

# PACKET CAPTURE: PACKET GENERATION

Byeong-hee Roh

Dept. of Software and Computer Engineering

Ajou University



변화와 융합의 중심  
**MMcN**

멀티미디어 융합 연구실

Mobile Multimedia convergene Network Lab.  
<http://mmcn.ajou.ac.kr>



# Contents

---

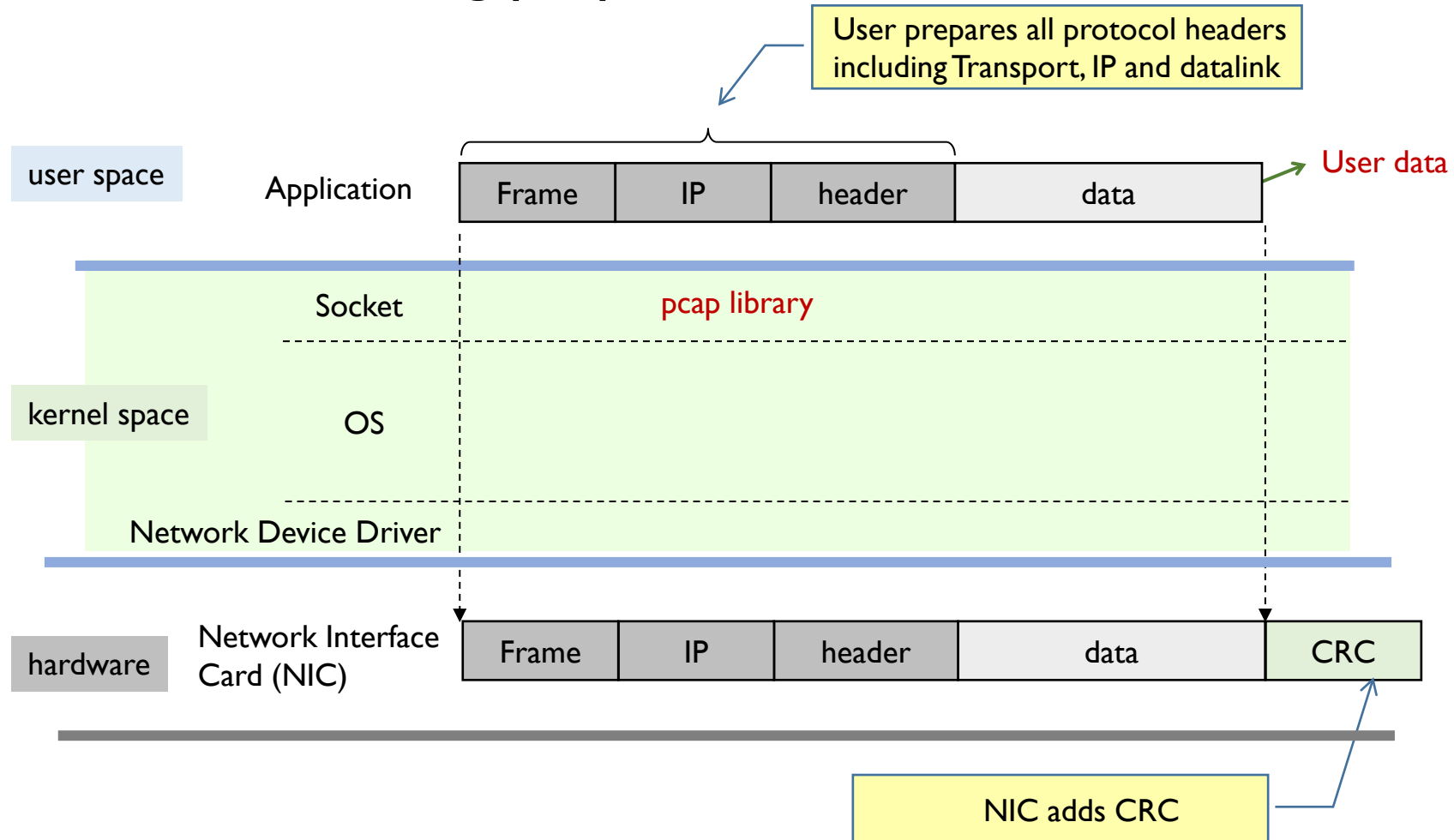
- ❖ Libpcap/WinPcap Packet Generation Functions
- ❖ libnet
- ❖ Linux PF\_SOCKET
- ❖ Libpcap Packet Generation Example
- ❖ ARP Example



# **LIBPCAP/WINPCAP PACKET GENERATION FUNCTIONS**

# Packet Generation

## ❖ Packet Generation using pcap



# Packet Generation – pcap\_inject( )

## ❖ pcap\_inject( )

- for libpcap
- to send a raw packet through the network interface

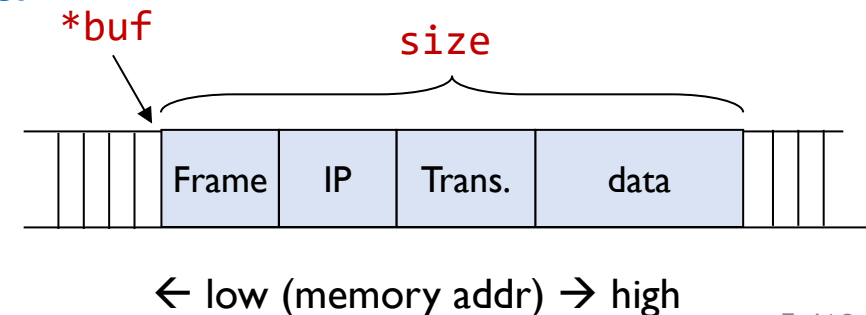
```
int  pcap_inject ( pcap_t      *p,  
                  const void  *buf,  
                  size_t      size);
```

### ▪ arguments

- **buf** – pointer to the packet including the link-layer header
- **size** – the number of bytes in the packet.

### ▪ return value

- the number of bytes written on success
- -1 on failure



# Packet Generation – pcap\_inject( )

## ❖ Example

```
pcap_t *handle;

// Retrieve the device list
pcap_findalldevs (&alldevs, errbuf);

// Open the adapter
handle = pcap_open_live( d->name, 65536, 1, 1000, errbuf);

// Prepare a packet
const char packet[] = { /* packet data*/ };

int bytes_sent = pcap_inject(handle, packet, sizeof(packet));

if (bytes_sent == -1) {
    fprintf(stderr, "Error sending packet: %s\n", pcap_geterr(handle));
} else {
    printf("Sent %d bytes... success !!! \n", bytes_sent);
}
pcap_close(handle);
```

# Packet Generation – pcap\_sendpacket( )

## ❖ pcap\_sendpacket( )

- to send a raw packet through the network interface

```
int    pcap_sendpacket (    pcap_t    *p,  
                           const u_char *buf,  
                           int         size);
```

- arguments
  - `buf` – pointer to the packet including the link-layer header
  - `size` – the number of bytes in the packet.
- return value
  - `0` on success
    - only the difference from `pcap_inject( )`
  - `PCAP_ERROR (-1)` on failure

# Packet Generation – pcap\_sendpacket( )

## ❖ Example

```
pcap_t *handle;

// Retrieve the device list
pcap_findalldevs (&alldevs, errbuf);

// Open the adapter
handle = pcap_open_live( d->name, 65536, 1, 1000, errbuf);

// Prepare a packet
const char packet[] = { /* packet data*/ };

int result = pcap_sendpacket(handle, (const u_char *)packet, sizeof(packet));
if (result != 0 ) {
    fprintf(stderr, "Error sending packet: %s\n", pcap_geterr(handle));
} else {
    printf("Packet sent success !!! \n");
}
pcap_close(handle);
```



# Packet Generation – Compariosn

## ❖ pcap\_inject vs. pcap\_sendpacket( )

| Feature               | pcap_inject                                                                      | pcap_sendpacket                                                                                                               |
|-----------------------|----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Return Value          | Number of bytes sent                                                             | Success/failure status (0/-1)                                                                                                 |
| Packet Size Type      | size_t                                                                           | int                                                                                                                           |
| Success/Failure Check | Can indirectly verify through bytes sent                                         | Directly indicates success or failure                                                                                         |
| Use Case              | When the exact number of bytes sent needs to be verified                         | Used for simple packet transmission                                                                                           |
| Compatibility         | Mostly used in standard libpcap environments like Linux                          | More common in Windows/WinPcap environments                                                                                   |
| Size limitation       | No limitation to packet size upto size_t (but, depends on MTU, about 1500 bytes) | Since the packet size is passed as <b>**int**</b> , there may be restrictions when sending large packets (up to 2GB or more). |

# Packet Generation – Queue Management

## ❖ Other related functions

- to allocate a send queue

```
pcap_send_queue* pcap_sendqueue_alloc ( u_int memsize );
```

- to add a packet at the end of the send queue

```
int pcap_sendqueue_queue ( pcap_send_queue* queue,  
                           const struct pcap_pkthdr* pkt_header,  
                           const u_char* pkt_data);
```

- to send a queue of raw packets to the network

```
u_int pcap_sendqueue_transmit ( pcap_t* p,  
                                pcap_send_queue* queue,  
                                int sync);
```

- to destroy a send queue

```
void pcap_sendqueue_destroy ( pcap_send_queue* queue );
```

# Packet Generation – Example

## ❖ npcap example – sendpack.c

- in Linux, change the line 131 as

```
#ifdef _WIN32
    *(u_long *)(packet + 14 + 12) = ((struct sockaddr_in *)(addr->addr))->sin_addr.S_un.S_addr;
#else
    *(u_long *)(packet + 14 + 12) = ((struct sockaddr_in *)(addr->addr))->sin_addr.s_addr;;
#endif
```

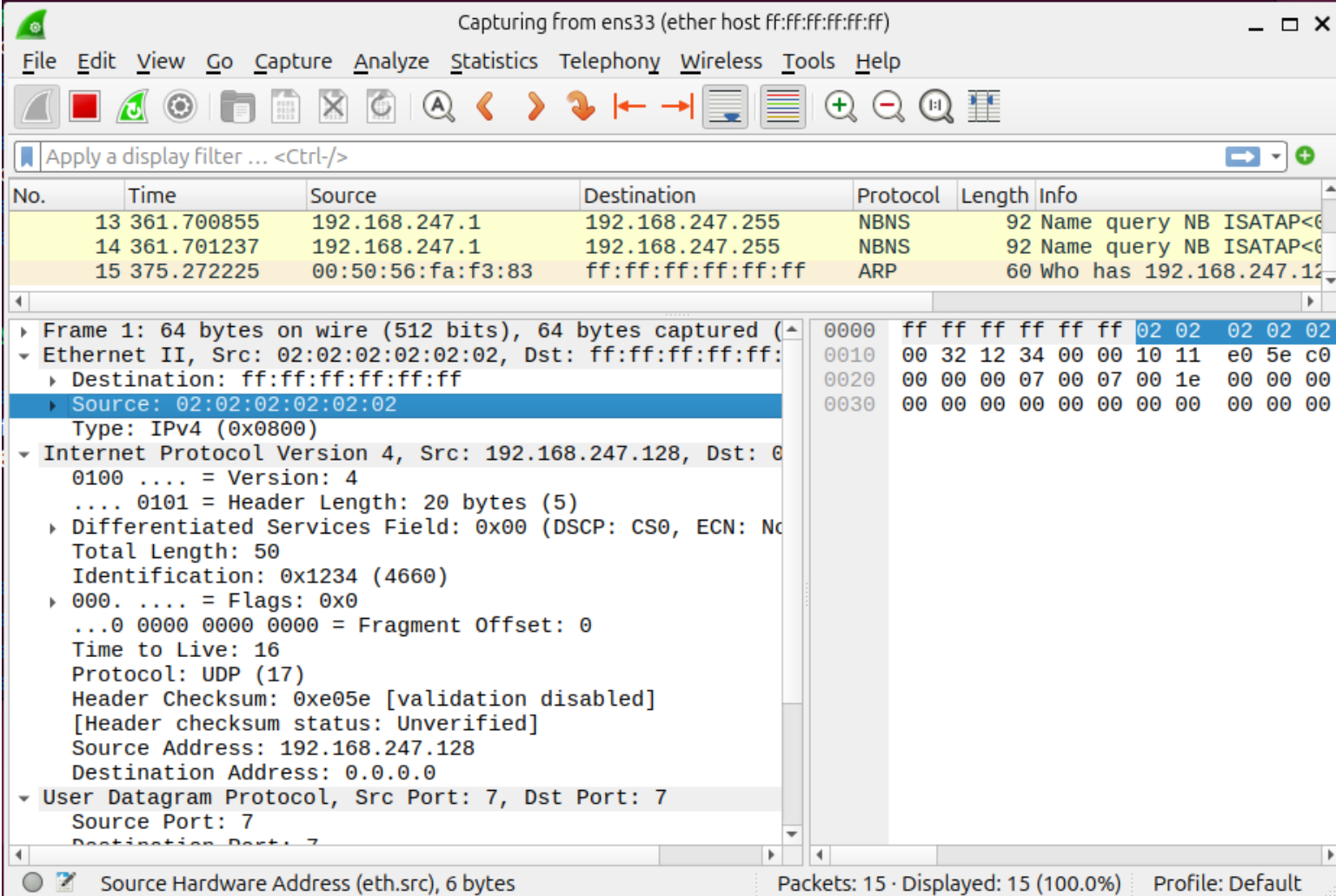
- usage
  - `printf("usage: %s <interface>", argv[0]);`

## ❖ Prctice

- add routines to select interface from the list of all device interfaces

# Packet Generation – Example

```
$ gcc sendpack.c -lpcap
$ sudo ./a.out ens33
```



Capturing from ens33 (ether host ff:ff:ff:ff:ff:ff)

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

| No. | Time       | Source            | Destination       | Protocol | Length | Info                   |
|-----|------------|-------------------|-------------------|----------|--------|------------------------|
| 13  | 361.700855 | 192.168.247.1     | 192.168.247.255   | NBNS     | 92     | Name query NB ISATAP<6 |
| 14  | 361.701237 | 192.168.247.1     | 192.168.247.255   | NBNS     | 92     | Name query NB ISATAP<6 |
| 15  | 375.272225 | 00:50:56:fa:f3:83 | ff:ff:ff:ff:ff:ff | ARP      | 60     | Who has 192.168.247.12 |

Frame 1: 64 bytes on wire (512 bits), 64 bytes captured (512 bits) on interface 0

Ethernet II, Src: 02:02:02:02:02:02, Dst: ff:ff:ff:ff:ff:ff

- Destination: ff:ff:ff:ff:ff:ff
- Source: 02:02:02:02:02:02
- Type: IPv4 (0x0800)

Internet Protocol Version 4, Src: 192.168.247.128, Dst: 0.0.0.0

- Version: 4
- Header Length: 20 bytes (5)
- Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
- Total Length: 50
- Identification: 0x1234 (4660)
- Flags: 0x0
- Fragment Offset: 0
- Time to Live: 16
- Protocol: UDP (17)
- Header Checksum: 0xe05e [validation disabled] [Header checksum status: Unverified]
- Source Address: 192.168.247.128
- Destination Address: 0.0.0.0

User Datagram Protocol, Src Port: 7, Dst Port: 7

Source Hardware Address (eth.src), 6 bytes

Packets: 15 · Displayed: 15 (100.0%) Profile: Default



# LIBNET

# libnet



## ❖ libnet

- <https://github.com/libnet/libnet>
- a C library that simplifies
  - the creation, manipulation, and transmission of network packets
- primarily used for low-level network programming
  - making it easier for tasks like penetration testing, network simulation, and custom protocol creation
  - providing a high-level API to construct packets for various network protocols,
    - such as TCP, UDP, ICMP, ARP, user-customized protocols, and more
    - with each control over packet structure and content
    - with handles the complexities of packet crafting and allows easy control over packet structure and content.

## ❖ Benefit of libnet

- complex network packet generation works
  - can be handled with simple function calls
- easily combine multiple protocols
  - to create multi-protocol packets
- can be used for various purposes
  - such as network analysis, testing, and attack simulation

## ❖ Limitation of libnet

- Due to the nature of low-level programming
  - it may be difficult for beginners to use
- There may be platform-specific dependencies
  - Some features may be limited for newer network protocols
- It is ideally targeted for packet generation and transmission
  - not providing packet capture



# Install libnet

---

## ❖ For linux

```
sudo apt-get install libnet-dev
```

## ❖ For macOS

```
brew install libnet
```

## ❖ For Windows

- It can write program with Npcap utilizing libnet source code



# libnet programming

## ❖ basic programming procedure using libnet (1/2)

- initialize libnet

```
l = libnet_init(LIBNET_RAW4, NULL, errbuf);
```

- create a packet (e.g. TCP)

```
u_long dest_ip = libnet_name2addr4(l, "192.168.1.1", LIBNET_RESOLVE);
```

```
// Build TCP packet
```

```
libnet_build_tcp(
```

```
    12345,
```

```
    // Source Port
```

```
    80,
```

```
    // Destination Port
```

```
    0x10000000,
```

```
    // Sequence Number
```

```
    0x20000000,
```

```
    // Acknowledgment Number
```

```
    TH_SYN,
```

```
    // Flags (SYN)
```

```
    32767,
```

```
    // Window Size
```

```
    0,
```

```
    // Checksum (0 for auto-calculation)
```

```
    0,
```

```
    // Urgent Pointer
```

```
    LIBNET_TCP_H,
```

```
    // Total Length
```

```
    NULL,
```

```
    // Payload
```

```
    0,
```

```
    // Payload Size
```

```
    1,
```

```
    // Libnet context
```

```
    0
```

```
    // Protocol Tag
```

```
);
```

# libnet programming

## ❖ basic programming procedure using libnet (2/2)

- send packet

```
res= libnet_write(l)
if ( res == -1)
    fprintf(stderr, "libnet_write() failed: %s\n", libnet_geterror(l));
```

- clean up

```
libnet_destroy(l);
```

- compile

```
gcc <program file> -lnet
```

# Sample

## ❖ icmp\_echo\_cq.c

- [https://github.com/libnet/libnet/blob/master/sample/icmp\\_echo\\_cq.c](https://github.com/libnet/libnet/blob/master/sample/icmp_echo_cq.c)

```
$ gcc icmp_echo_cq.c -lnet
$ sudo ./a.out -s 192.168.0.10 -d 192.168.0.1
libnet 1.1 packet shaping: ICMP[RAW using context queue]
Wrote 28 byte ICMP packet from context "echo 9"; check the wire.
Wrote 28 byte ICMP packet from context "echo 8"; check the wire.
Wrote 28 byte ICMP packet from context "echo 7"; check the wire.
Wrote 28 byte ICMP packet from context "echo 6"; check the wire.
Wrote 28 byte ICMP packet from context "echo 5"; check the wire.
Wrote 28 byte ICMP packet from context "echo 4"; check the wire.
Wrote 28 byte ICMP packet from context "echo 3"; check the wire.
Wrote 28 byte ICMP packet from context "echo 2"; check the wire.
Wrote 28 byte ICMP packet from context "echo 1"; check the wire.
Wrote 28 byte ICMP packet from context "echo 0"; check the wire.
```

Wi-Fi (icmp)에서 캡처 중

파일(F) 편집(E) 보기(V) 이동(G) 캡처(C) 분석(A) 통계(S) 전화(Y) 무선(W) 도구(T) 도움말(H)

표시 필터 적용 ... <Ctrl-/>

| No. | Time     | Source      | Destination  | Protocol | Len | Info                                 |
|-----|----------|-------------|--------------|----------|-----|--------------------------------------|
| 15  | 0.002142 | 192.168.0.1 | 192.168.0.10 | ICMP     | 52  | Echo (ping) reply id=0x0042, seq=66. |
| 16  | 0.002142 | 192.168.0.1 | 192.168.0.10 | ICMP     | 52  | Echo (ping) reply id=0x0042, seq=66. |
| 17  | 0.007113 | 192.168.0.1 | 192.168.0.10 | ICMP     | 52  | Echo (ping) reply id=0x0042, seq=66. |
| 18  | 0.007300 | 192.168.0.1 | 192.168.0.10 | ICMP     | 52  | Echo (ping) reply id=0x0042, seq=66. |
| 19  | 0.007699 | 192.168.0.1 | 192.168.0.10 | ICMP     | 52  | Echo (ping) reply id=0x0042, seq=66. |
| 20  | 0.007699 | 192.168.0.1 | 192.168.0.10 | ICMP     | 52  | Echo (ping) reply id=0x0042, seq=66. |

> Frame 1: 42 bytes on wire (336 bits), 42 bytes captured  
> Ethernet II, Src: 14:18:c3:7e:dc:0e, Dst: 70:5d:cc:92:5d:00  
> Internet Protocol Version 4, Src: 192.168.0.10, Dst: 192.168.0.10

▼ Internet Control Message Protocol

- Type: 8 (Echo (ping) request)
- Code: 0
- Checksum: 0xf77b [correct]  
[Checksum Status: Good]
- Identifier (BE): 66 (0x0042)
- Identifier (LE): 16896 (0x4200)
- Sequence Number (BE): 66 (0x0042)
- Sequence Number (LE): 16896 (0x4200)

> [No response seen]

0000 70 5d cc 92 5d ac 14 18 c3 7e dc 0e 08 00 45 00 p  
0010 00 1c e8 c4 00 00 3f 01 00 00 c0 a8 00 0a c0 a8  
0020 00 01 08 00 f7 7b 00 42 00 42

http: Wi-Fi: <live capture in progress> 패킷: 20 프로파일: Default



# LINUX PF\_SOCKET



# PF\_PACKET

---

## ❖ PF\_PACKET

- a socket type used on Linux for packet capture
  - allows to access raw packets directly from the network interface
- libpcap uses PF\_PACKET also, providing
  - a high-level library that simplifies packet capturing
  - a cleaner API to interact with various network interfaces

# PF\_PACKET

## ❖ Usage procedure (1/2)

### ▪ socket creation

```
int sock = socket(PF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
```

### ▪ interface binding

```
struct ifreq ifr;  
struct sockaddr_ll sll;  
  
strncpy(ifr.ifr_name, "eth0", IFNAMSIZ); // set interface name  
res = ioctl(sock, SIOCGIFINDEX, &ifr) ;  
  
memset(&sll, 0, sizeof(sll));  
sll.sll_family    = AF_PACKET;  
sll.sll_ifindex   = ifr.ifr_ifindex; // 인터페이스 인덱스  
sll.sll_protocol  = htons(ETH_P_ALL);  
  
bind(sock, (struct sockaddr *)&sll, sizeof(sll))
```



# PF\_PACKET

---

## ❖ Usage procedure (2/2)

### ▪ packet generation

```
unsigned char *packet;  
  
int len = sendto(sock, packet, sizeof(packet), 0, (struct sockaddr *)&sll, sizeof(sll));
```

### ▪ packet reception

```
unsigned char buffer[2048];  
  
int len = recvfrom(sock, buffer, sizeof(buffer), 0, NULL, NULL);
```

# PF\_PACKET

## ❖ Example (1/3)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/socket.h>
#include <net/if.h>
#include <linux/if_packet.h>
#include <netinet/ether.h>
#include <arpa/inet.h>
#include <sys/ioctl.h>

int main() {
    int sock;
    struct ifreq ifr;
    struct sockaddr_ll sll;
    unsigned char buffer[2048];
```

```
// 소켓 생성
sock = socket(PF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
if (sock == -1) {
    perror("Socket creation failed");
    return -1;
}

// 네트워크 인터페이스 가져오기
strncpy(ifr.ifr_name, "ens33", IFNAMSIZ);
if (ioctl(sock, SIOCGIFINDEX, &ifr) == -1) {
    perror("Failed to get interface index");
    close(sock);
    return -1;
}

// 소켓 주소 설정
memset(&sll, 0, sizeof(sll));
sll.sll_family = AF_PACKET;
sll.sll_ifindex = ifr.ifr_ifindex;
sll.sll_protocol = htons(ETH_P_ALL);
```



# PF\_PACKET

## ❖ Example (2/3)

```
if (bind(sock, (struct sockaddr *)&sll, sizeof(sll)) == -1) {
    perror("Bind failed");
    close(sock);
    return -1;
}

unsigned char packet[64] = {
    0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, // 목적지 MAC (브로드캐스트)
    0x00, 0x0C, 0x29, 0x6B, 0x1A, 0x0E, // 출발지 MAC
    0x08, 0x00, // 이더넷 타입 (IPv4)
};

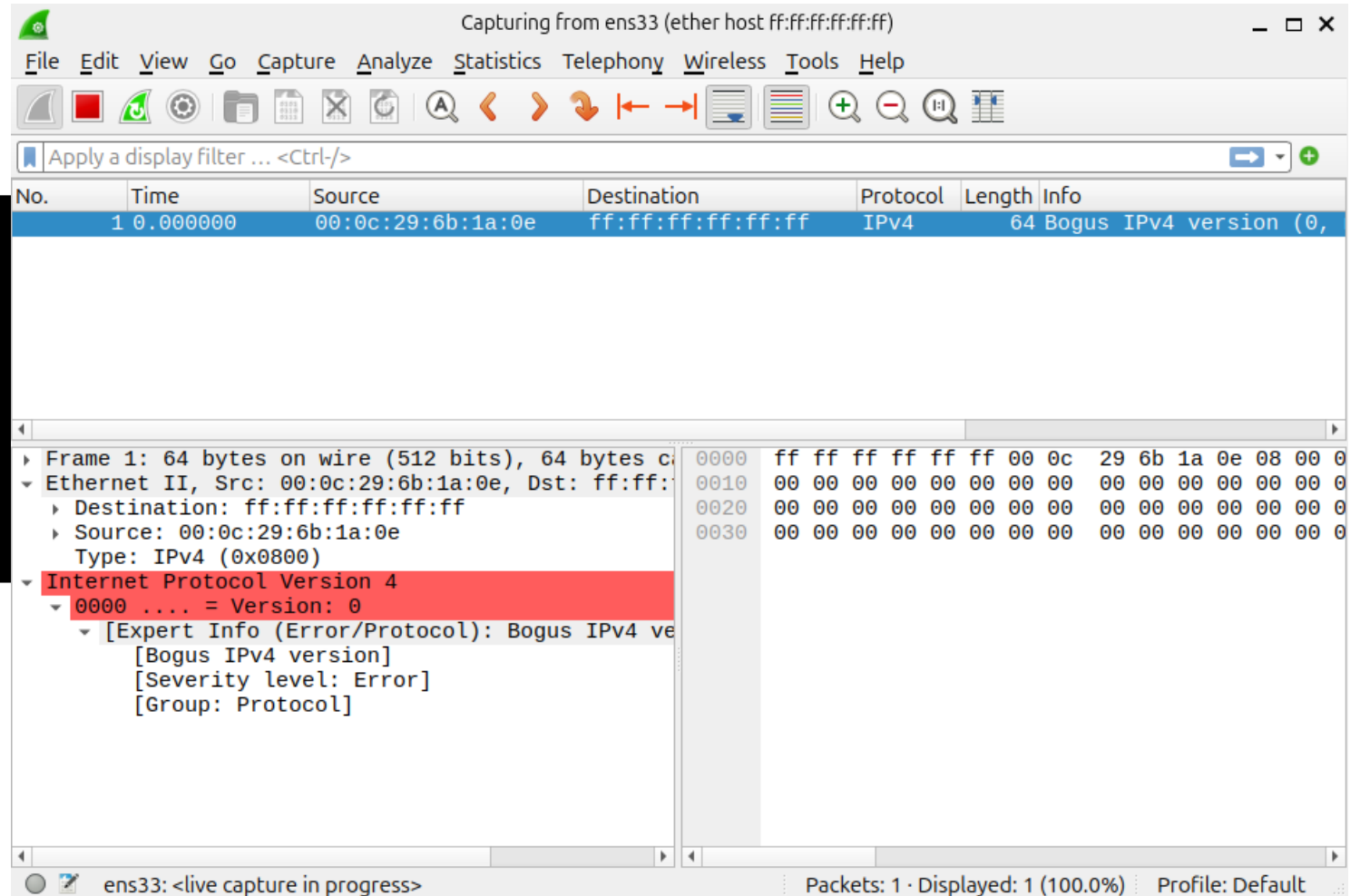
// 패킷 수신
int len = sendto(sock, packet, sizeof(packet), 0, (struct sockaddr *)&sll, sizeof(sll));
printf("Packet sent successfully\n");
}

// 패킷 수신
printf("Listening for packets...\n");
int len = recvfrom(sock, buffer, sizeof(buffer), 0, NULL, NULL);
printf("Received packet of length %d\n", len);
close(sock);
return 0;
}
```

# PF\_PACKET

## ❖ Example (3/3)

```
$ gcc pf_ex.c
$ sudo ./a.out
Packet sent successfully
Listening for packets...
```

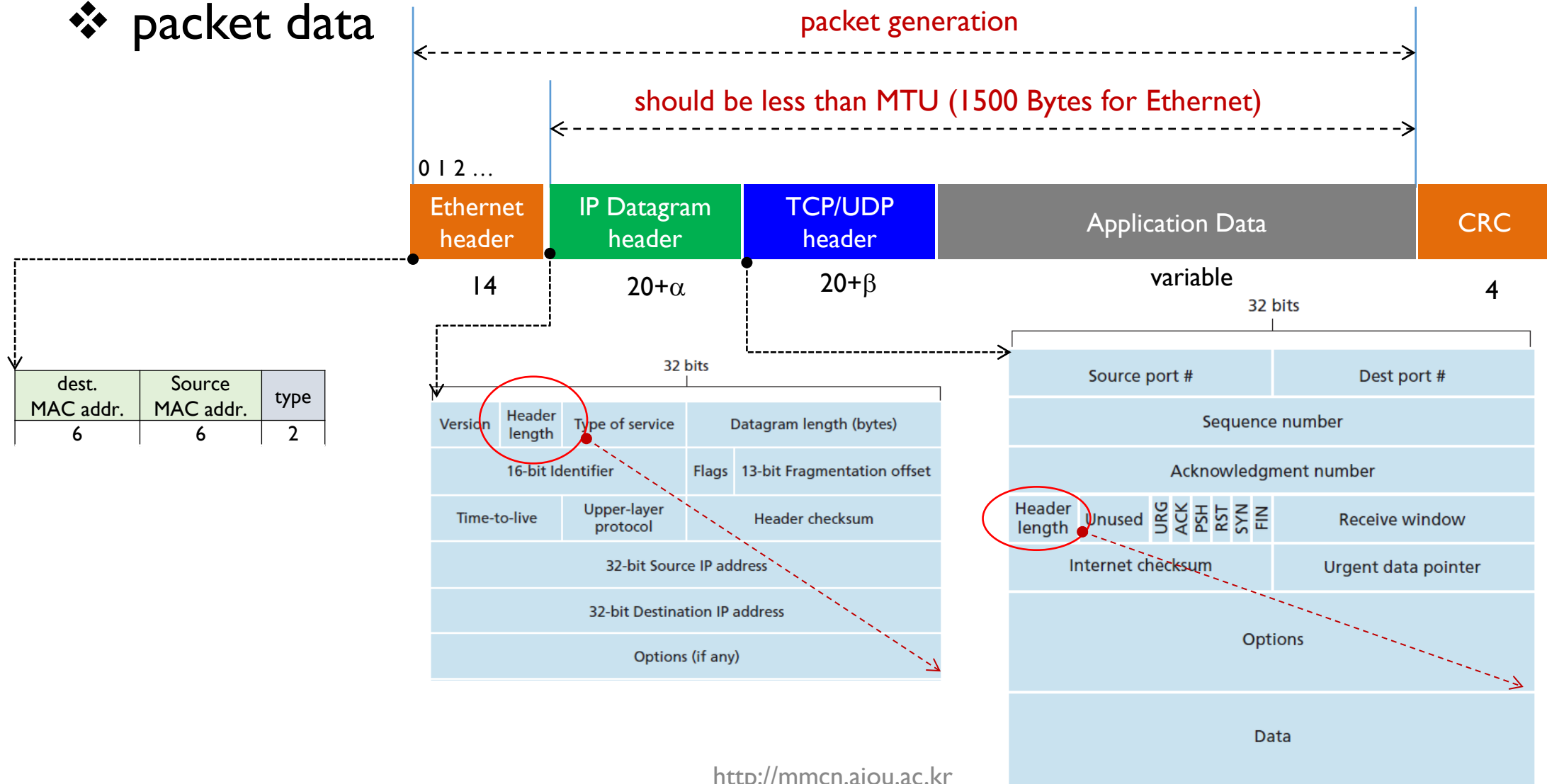


# **LIBPCAP PACKET GENERATION EXAMPLE**



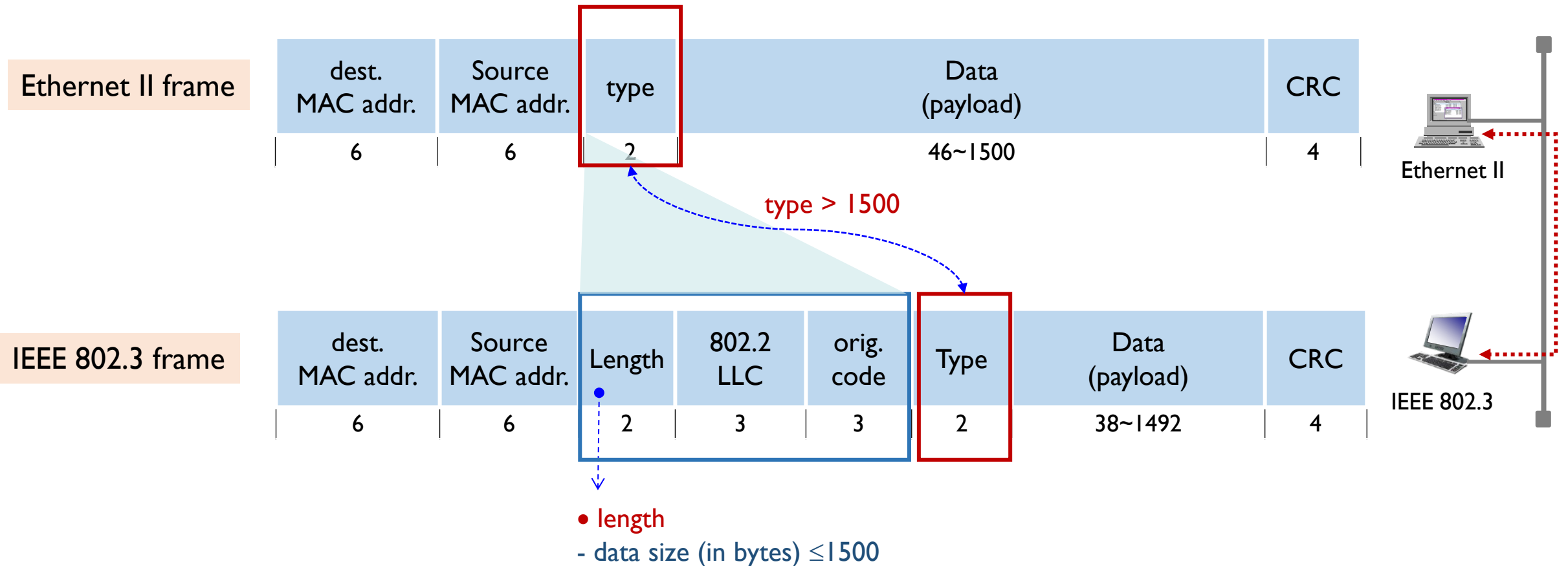
# Packet Generation Data Structure

## ❖ packet data



# Ethernet Header Generation (1/3)

## ❖ Ethernet II and IEEE 802.3 Ethernet frame structure



# Ethernet Header Generation (2/3)

## ❖ Ethernet header data structure

- by referring to `/usr/include/linux/if_ether.h` in linux

```
#define      MAC_ADDR_LEN      6      // length of MAC address (in bytes)
#define      ETHERTYPE_IP      0x0800 // IP protocol
#define      ETHERTYPE_ARP      0x0806 // ARP protocol

// Ethernet Header Structure
struct eth_header {
    uint8_t  dst_mac[6]; // Destination MAC Address
    uint8_t  src_mac[6]; // Source MAC Address
    uint16_t ethertype;   // EtherType (e.g., IPv4: 0x0800)
};
```

| dst_mac            | src_mac             | ethertype |                   |     |
|--------------------|---------------------|-----------|-------------------|-----|
| dest.<br>MAC addr. | Source<br>MAC addr. | type      | Data<br>(payload) | CRC |
| 6                  | 6                   | 2         | 46~1500           | 4   |

# Ethernet Header Generation (3/3)

## ❖ Example code to generate Ethernet header

```
// Ethernet header fields example
uint8_t dst_mac[6] = { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF }; // Broadcast MAC
uint8_t src_mac[6] = { 0x0a, 0x0b, 0x0c, 0x01, 0x02, 0x03 }; // Source MAC (temp.)
uint16_t etype = 0x0800;

// Ethernet header generation
void get_header_ethernet(struct eth_header *eth) {

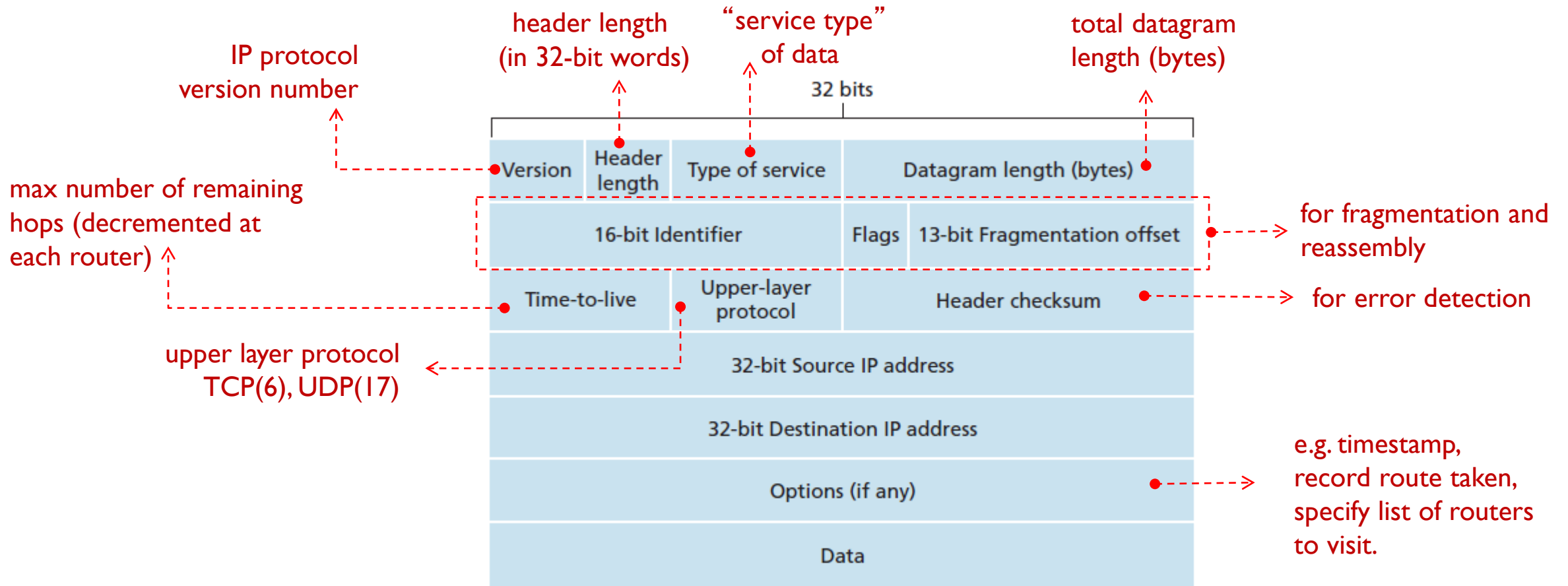
    // copy destination mac to ethernet header
    memcpy(eth->dst_mac, dst_mac, 6);

    // copy source mac to ethernet header
    memcpy(eth->src_mac, src_mac, 6);

    // EtherType (network byte order)
    eth->ethertype = htons(etype);
}
```

# IP Header Generation (1/4)

## ❖ IP datagram header format





# IP Header Generation (2/4)

- ❖ IP datagram header data structure
  - by referring to `/usr/include/linux/ip.h` in linux

```
// IP header structure (IPv4)
struct ip_header {
    uint8_t  ihl : 4;           // Header Length
    uint8_t  version : 4;       // IP Version
    uint8_t  tos;               // Type of Service
    uint16_t total_length;       // Total Length
    uint16_t id;                // Identification
    uint16_t fragment_offset;    // Fragment Offset
    uint8_t  ttl;               // Time to Live
    uint8_t  protocol;          // Protocol (TCP: 6, UDP: 17)
    uint16_t checksum;          // Header Checksum
    uint32_t src_ip;             // Source IP Address
    uint32_t dst_ip;             // Destination IP Address
};
```

# IP Header Generation (3/4)

## ❖ Example code to generate IP header (1/2)

```
// IP header fields example
char    *sip = "192.168.0.110";
char    *dip = "192.168.0.120";
uint8_t protocol = IPPROTO_UDP;

void get_header_ip(struct ip_header* iph)
{
    uint32_t          src_ip, dst_ip;
    uint16_t          upp_len, checksum = 0;
    uint8_t           buffer[4096] = { 0 };

    // IP header fields example
    src_ip = inet_addr(sip);           // source IP address
    dst_ip = inet_addr(dip);           // destination IP address

    if (protocol == IPPROTO_TCP)
    {
        upp_len = 20 + sizeof(struct tcp_header) + strlen(app_data);
    }
    else
    {
        upp_len = 20 + sizeof(struct udp_header) + strlen(app_data);
    }
}
```

# IP Header Generation (4/4)

## ❖ Example code to generate IP header (2/2)

```
iph->version      = 4;           // IPv4
iph->ihl           = 5;           // Header Length (5 words), no option
iph->tos           = 0;           // Default TOS
iph->total_length  = htons(upp_len); // no option
iph->id            = htons(54321); // Example Identification
iph->fragment_offset = 0;         // No fragmentation
iph->ttl           = 64;          // Default TTL
iph->protocol      = protocol;    // Protocol (TCP, UDP, ICMP, etc)
iph->checksum      = 0;           // Will be calculated later
iph->src_ip        = src_ip;      // Convert to network byte order
iph->dst_ip        = dst_ip;      // Convert to network byte order

memcpy(buffer, iph, sizeof(struct ip_header));

// checksum calculation
iph->checksum      = calculate_checksum(buffer, sizeof(struct ip_header));
}
```

# Checksum Calculation

```
// Checksum Calculation
uint16_t calculate_checksum(uint8_t* buffer, int length) {
    uint32_t    checksum= 0;
    uint16_t*    buffer_16        = (uint16_t*)buffer;

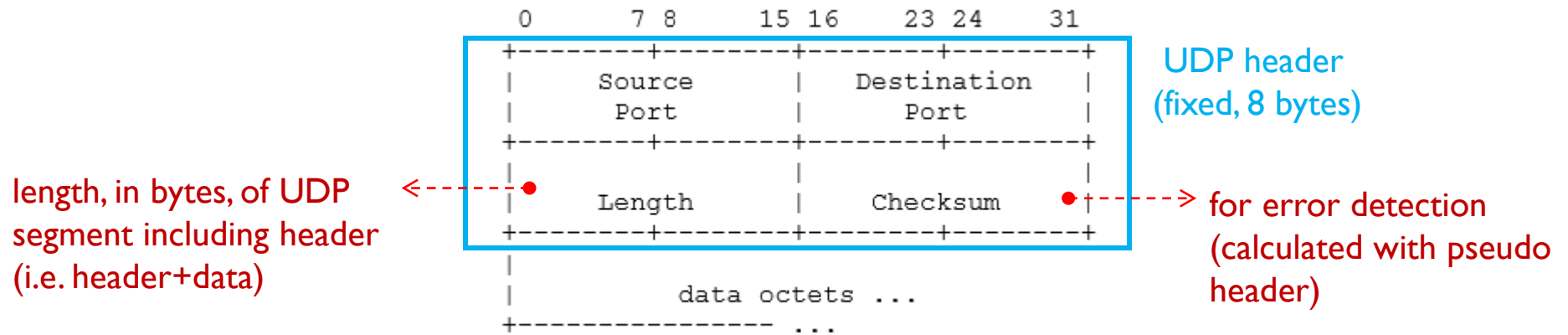
    // Padding zero for odd number buffer
    if (length % 2 == 1) {
        buffer[length] = 0;
        length += 1;
    }

    while (length > 1) {
        checksum += *buffer_16++; // 2bytes
        length -= 2;
    }
    // Wrap around for carry over 16-bits length
    checksum = (checksum >> 16) + (checksum & 0xFFFF);

    // one's complement
    return ~checksum;
}
```

# UDP Header Generation (1/5)

## ❖ UDP header format



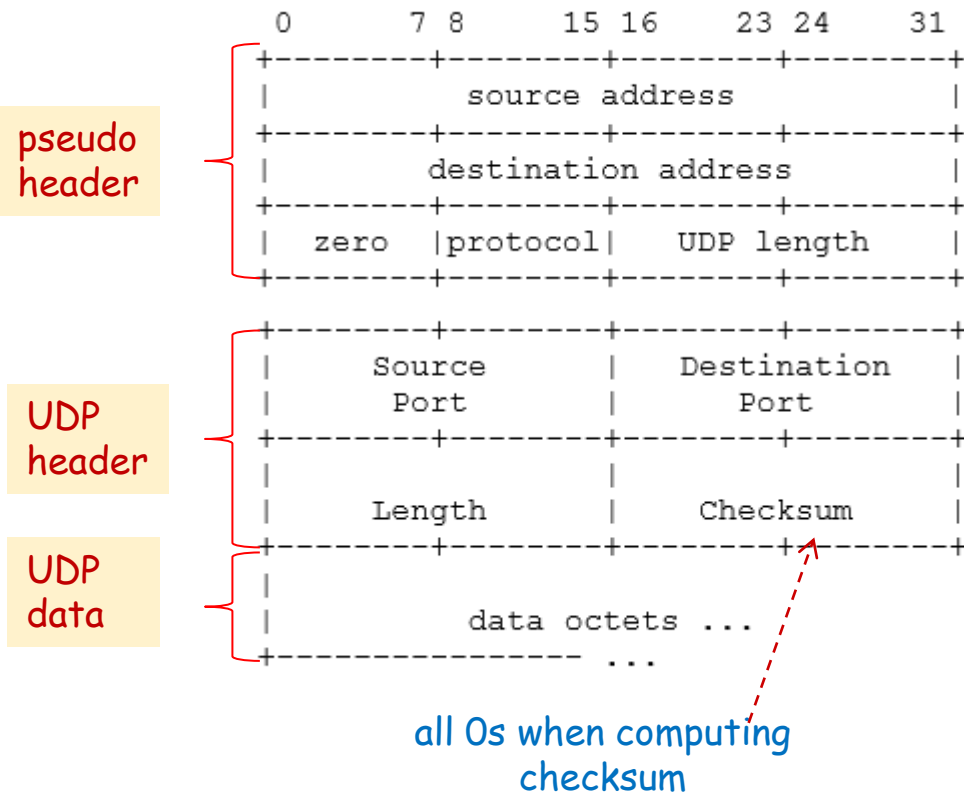
User Datagram Header Format

## ■ data structure

```
// UDP header structure
struct udp_header {
    uint16_t src_port;           // Source Port
    uint16_t dst_port;           // Destination Port
    uint16_t length;             // UDP Length
    uint16_t checksum;           // UDP Checksum
};
```

# UDP Header Generation (2/5)

## ❖ UDP pseudo header for checksum calculation



```
// Pseudo-header for checksum calculation
struct pseudo_header {
    uint32_t  src_addr; // Source IP Address
    uint32_t  dst_addr; // Destination IP Address
    uint8_t   zeros;    // zero bits
    uint8_t   protocol; // protocol field in IP header
    uint16_t  length;    // length of header + data
};
```

# UDP Header Generation (3/5)

## ❖ example code to generate UDP header (1/2)

```
// UDP header fields example
uint16_t    src_port    = 9999;
uint16_t    dst_port    = 80;

// Application data
char        *app_data = "Hello World, I am the Hero !!!";

void    get_header_udp(struct udp_header* udph)
{
    struct pseudo_header    psh;
    uint32_t                checksum = 0;
    uint8_t                 buffer[4096] = { 0 };
    int                     hlen;

    // UDP header fields without checksum
    udph->src_port    = htons(src_port);    // Convert to network byte order
    udph->dst_port    = htons(dst_port);    // Convert to network byte order
    udph->length      = htons(sizeof(struct udp_header) + strlen(app_data));
    udph->checksum    = 0;                  // Optional for UDP, can be left as 0
}
```

# UDP Header Generation (4/5)

## ❖ example code to generate UDP header (2/2)

```
// Get pseudo header
get_header_pseudo(&psh);

// Concatenate packets Pseudo, UDP and App data into one
memcpy(buffer, &psh, sizeof(struct pseudo_header));
memcpy(buffer + sizeof(struct pseudo_header),
        udph, sizeof(struct udp_header));
memcpy(buffer + sizeof(struct pseudo_header) + sizeof(struct udp_header),
        app_data, strlen(app_data));

// UDP checksum calculation
hlen = (int)sizeof(struct pseudo_header) + (int)sizeof(struct udp_header)
        + (int)strlen(app_data);

udph->checksum = calculate_checksum(buffer, hlen);

}
```



# UDP Header Generation (5/5)

## ❖ example code to generate Pseudo header

```
void get_header_pseudo(struct pseudo_header *psh)
{
    uint32_t          src_ip, dst_ip;

    // IP header fields example
    src_ip = inet_addr(sip);           // source IP address
    dst_ip = inet_addr(dip);           // destination IP address

    // pseudo header
    psh->src_addr    = src_ip;
    psh->dst_addr    = dst_ip;
    psh->zeros       = 0;
    psh->protocol    = protocol;
    if (protocol == IPPROTO_TCP)
        psh->length = htons(sizeof(struct tcp_header) + strlen(app_data));
    else
        psh->length = htons(sizeof(struct udp_header) + strlen(app_data));
}
```

# UDP Packet Generation – All Together (1/2)

```
void generate_packet_tcp_udp(u_char* packet, int *pklen) {
    char                sendbuf[4096] = { 0 };
    struct eth_header   *eth;
    struct ip_header    *iph;
    struct udp_header    *udph;
    char                *udata;

    // Concatenate Ethernet, IP, UDP and Data into sendbuf
    eth = (struct eth_header*)sendbuf;
    iph = (struct ip_header*)(sendbuf + sizeof(struct eth_header));
    udph = (struct udp_header*)(sendbuf + sizeof(struct eth_header)
                                + sizeof(struct ip_header));
    udata = (char*)(sendbuf + sizeof(struct eth_header)
                    + sizeof(struct ip_header) + sizeof(struct udp_header));
    *pklen = (int)sizeof(struct eth_header) + (int)sizeof(struct ip_header)
            + (int)sizeof(struct udp_header) + (int)strlen(udata);

    strcpy(udata, app_data);           // Application data
    get_header_udp(udph);               // UDP header
    get_header_ip(iph);                 // ip header
    get_header_ethernet(eth);           // Ethernet header
}
```

# UDP Packet Generation – All Together (2/2)

```
int main()
{
    pcap_if_t          *alldevs;
    pcap_if_t          *d;
    int                 i, dnum, snum;

    pcap_findalldevs(&alldevs, errbuf);    // Retrieve the device list

    ifprint0(alldevs, &dnum);              // Scan the list printing every entry

    // Select the interface for handling packets
    printf("Enter the interface number (1-%d):", dnum);
    scanf("%d", &snum);

    for (d = alldevs, i = 0; i < snum - 1; d = d->next, i++);

    adhandle = pcap_open_live(d->name, 65536, 1, 1000, effbuf); // Open the adapter

    generate_packet_tcp_udp(packet, &pklen);                    // UDP segment generation

    pcap_sendpacket(adhandle, packet, pklen);                    // Send packet
}
```

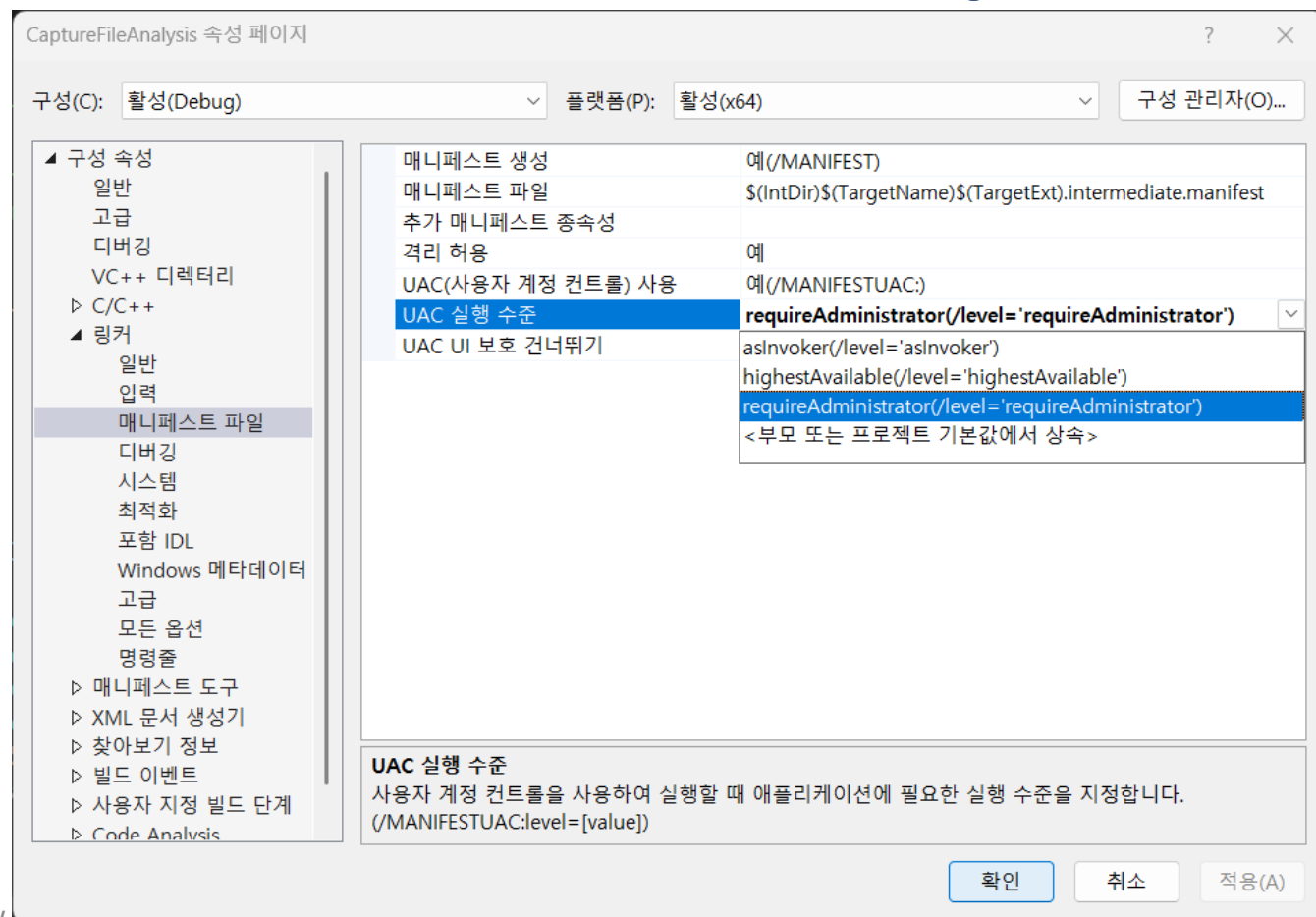
# Tips

## ❖ For Windows

- to run debug-level execution, set UAC level needs to be Admin. Privilege

## ❖ For Linux or macOS

- run as sudo...



Capturing from ens33 (port 9999)

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

| No. | Time     | Source        | Destination   | Protocol | Length | Info             |
|-----|----------|---------------|---------------|----------|--------|------------------|
| 1   | 0.000000 | 192.168.0.110 | 192.168.0.120 | UDP      | 72     | 9999 → 80 Len=30 |

Frame 1: 72 bytes on wire (576 bits), 72 bytes captured (576 bits)

- Ethernet II, Src: 0a:0b:0c:01:02:03, Dst: ff:ff:ff:ff:ff:ff
  - Destination: ff:ff:ff:ff:ff:ff
  - Source: 0a:0b:0c:01:02:03
  - Type: IPv4 (0x0800)
- Internet Protocol Version 4, Src: 192.168.0.110, Dst: 192.168.0.120
  - 0100 .... = Version: 4
  - .... 0101 = Header Length: 20 bytes (5)
  - Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  - Total Length: 58
  - Identification: 0xd431 (54321)
  - 000. .... = Flags: 0x0
  - ...0 0000 0000 0000 = Fragment Offset: 0
  - Time to Live: 64
  - Protocol: UDP (17)
  - Header Checksum: 0x244b [correct]
  - [Header checksum status: Good]
  - [Calculated Checksum: 0x244b]
  - Source Address: 192.168.0.110
  - Destination Address: 192.168.0.120
- User Datagram Protocol, Src Port: 9999, Dst Port: 80
  - Source Port: 9999
  - Destination Port: 80
  - Length: 38
  - Checksum: 0x80a7 [correct]
  - [Checksum Status: Good]
  - [Stream index: 0]
  - [Timestamps]
  - UDP payload (30 bytes)
- Data (30 bytes)
  - Data: 48656c6c6f20576f726c642c204920616d20746865204865726f20212121
  - [Length: 30]

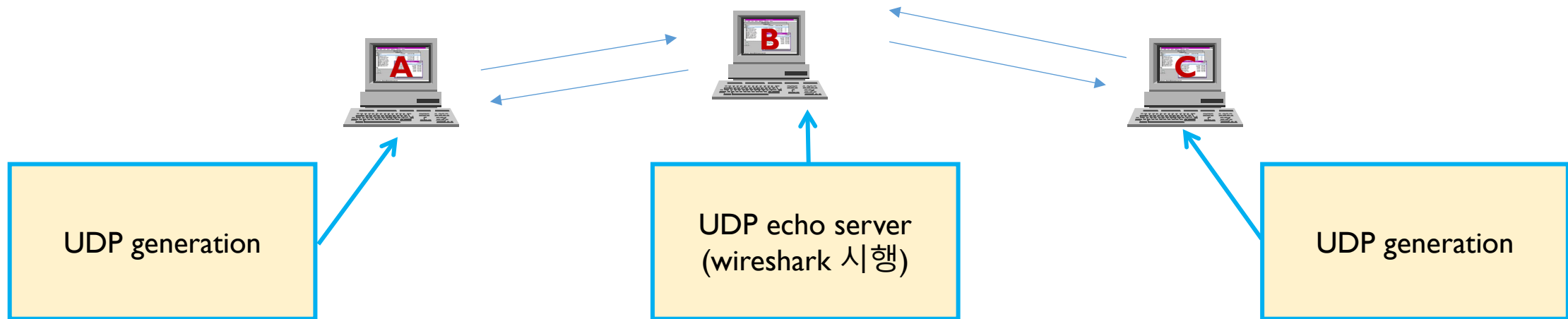
0000 ff ff ff ff ff ff 0a 0b 0c 01 02 03 08 00 45 00 .....E.  
0010 00 3a d4 31 00 00 40 11 24 4b c0 a8 00 6e c0 a8 ...:1..@. \$K...n..  
0020 00 78 27 0f 00 50 00 26 80 a7 48 65 6c 6c 6f 20 ..x'..P.& ..Hello  
0030 57 6f 72 6c 64 2c 20 49 20 61 6d 20 74 68 65 20 World, I am the  
0040 48 65 72 6f 20 21 21 21 Hero !!!

Data (data.data), 30 bytes

Packets: 1 · Displayed: 1 (100.0%) Profile: Default

# Practice #1

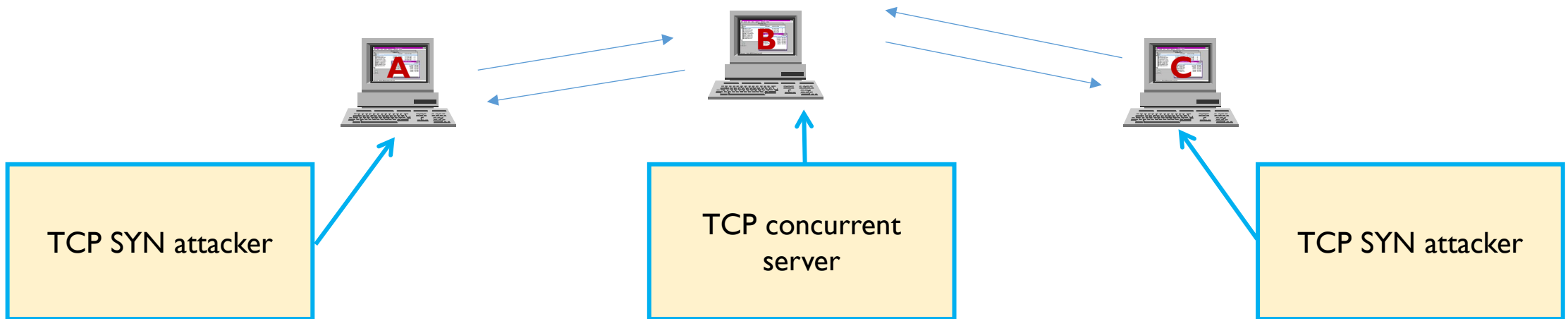
## ❖ UDP echo client server



# Practice #2

## ❖ TCP SYN Flooding

- UDP generation program을 활용하여 TCP generation program을 작성
- SYN segment를 연속으로 서버에 전송





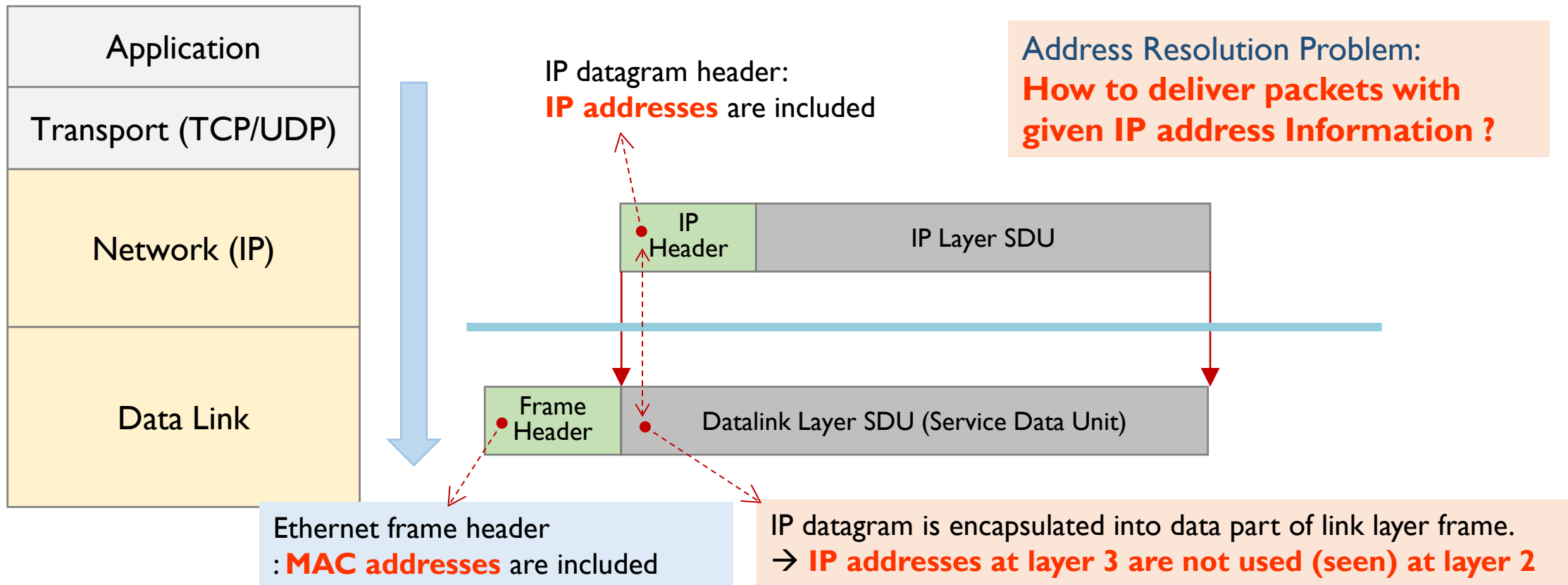
# ARP EXAMPLE



# ARP (Address Resolution Protocol)

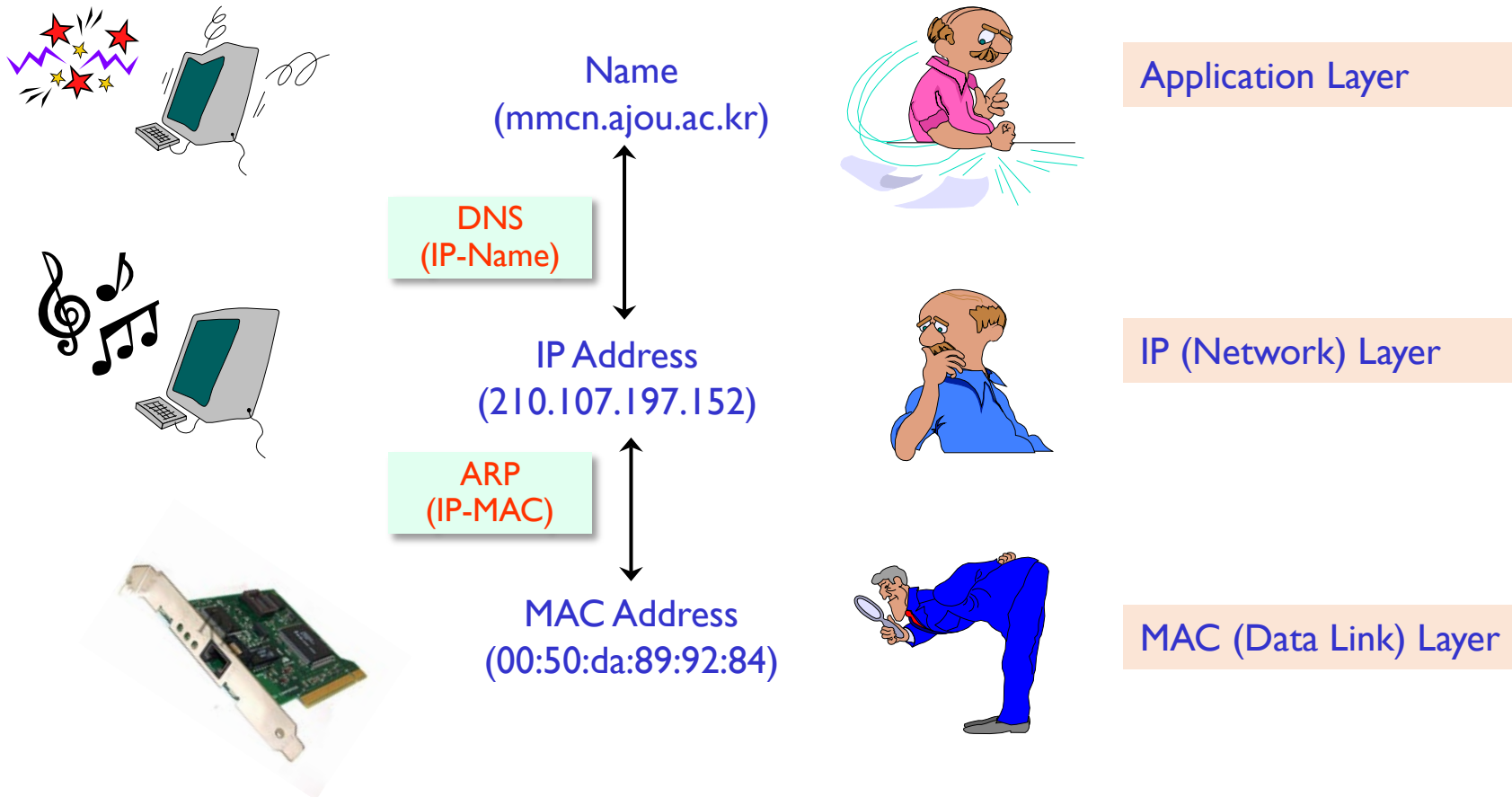
## ❖ Address Resolution Problem

- IP address at layer 3 is encapsulated (not seen) at datalink layer



# ARP

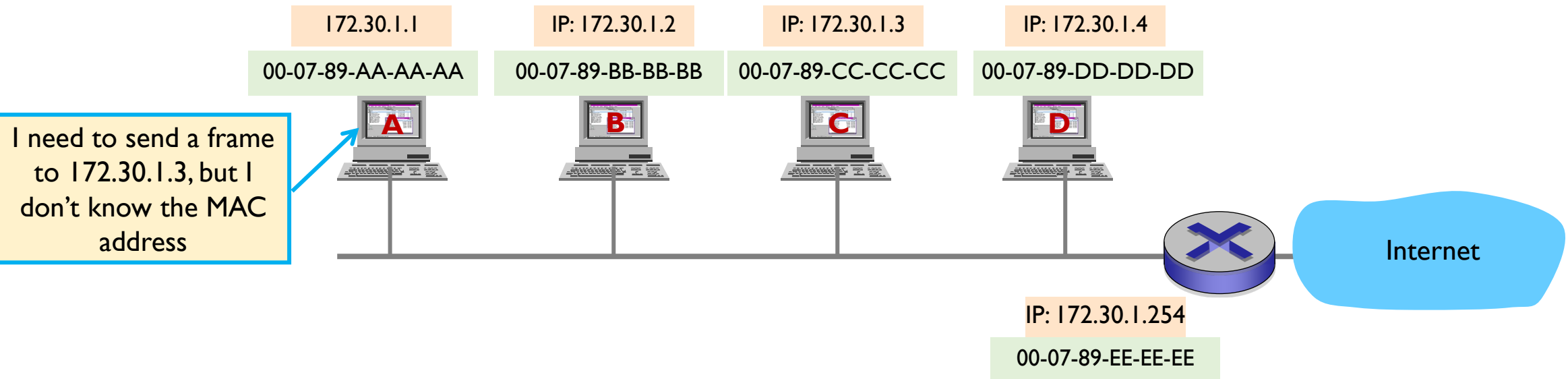
## ❖ Resolution Protocols for Name and Address Conversion



# ARP Operation

## ❖ ARP Operation Environments

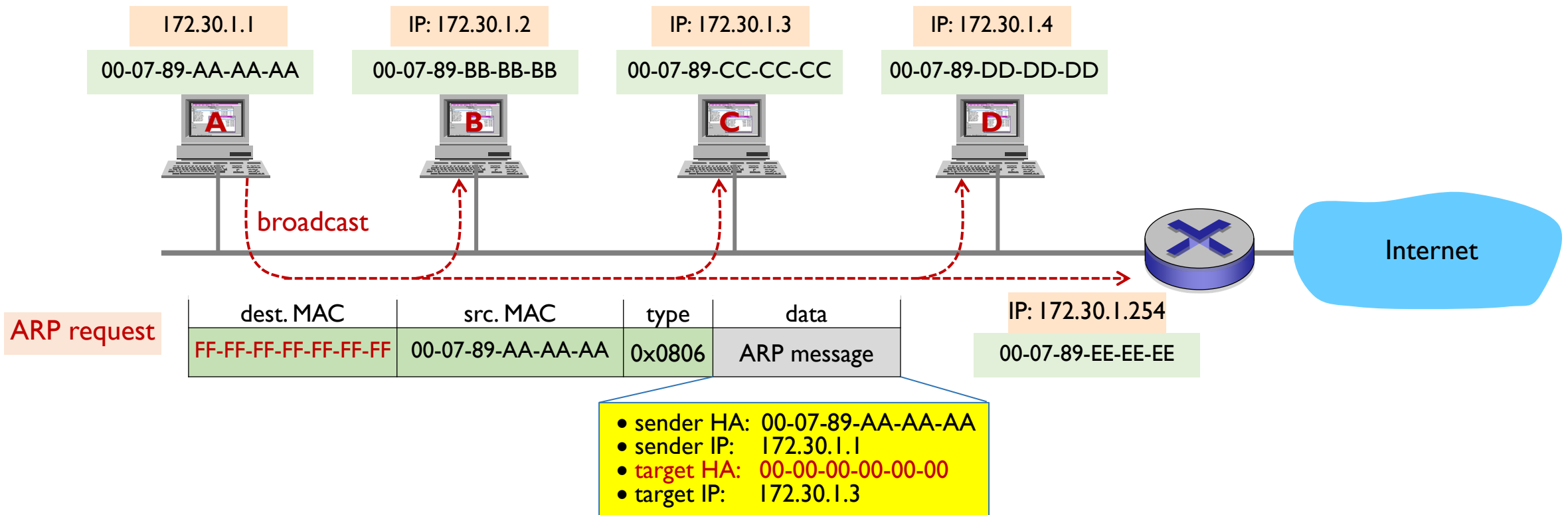
- to send a frame on a LAN media,
  - hosts must know destination MAC address
- host A wants to deliver a frame to host C
- host A knows
  - only host C's IP address, but, host C's MAC address is not known)



# ARP Operation

## ❖ ARP Request

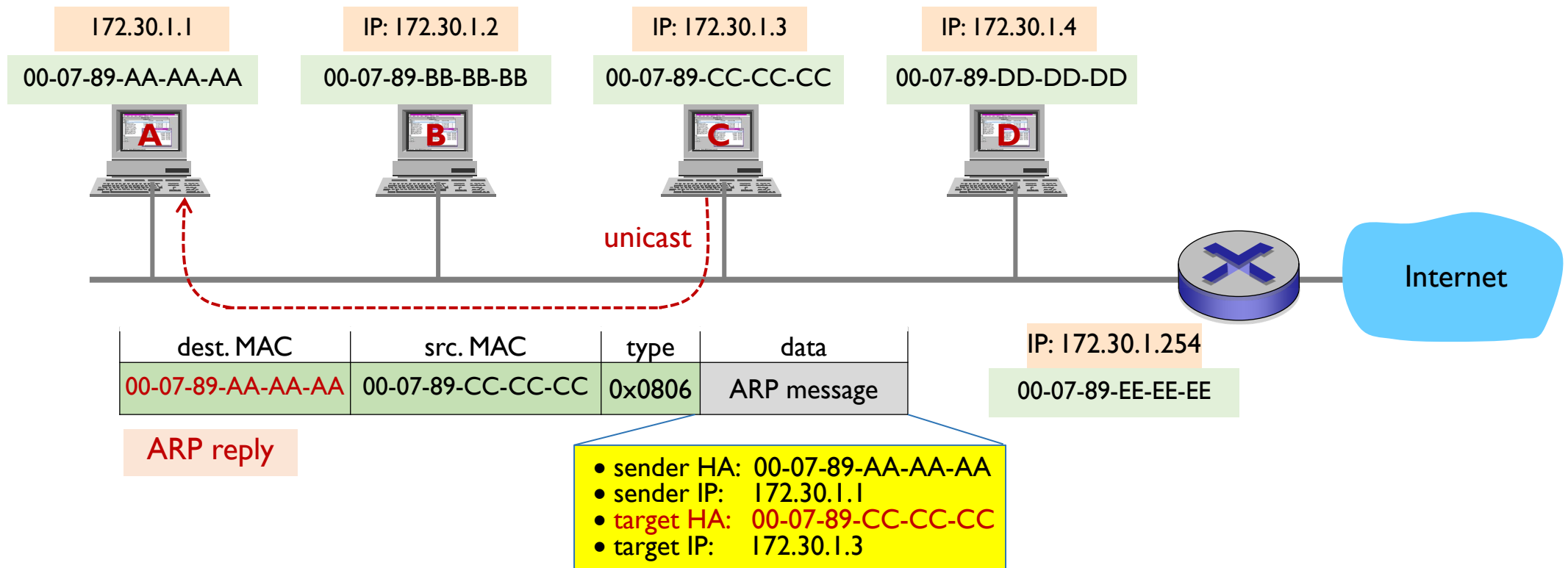
- host A broadcasts ARP Request message to all hosts in its network,
  - in which host C's IP address is included



# ARP Operation

## ❖ ARP Reply

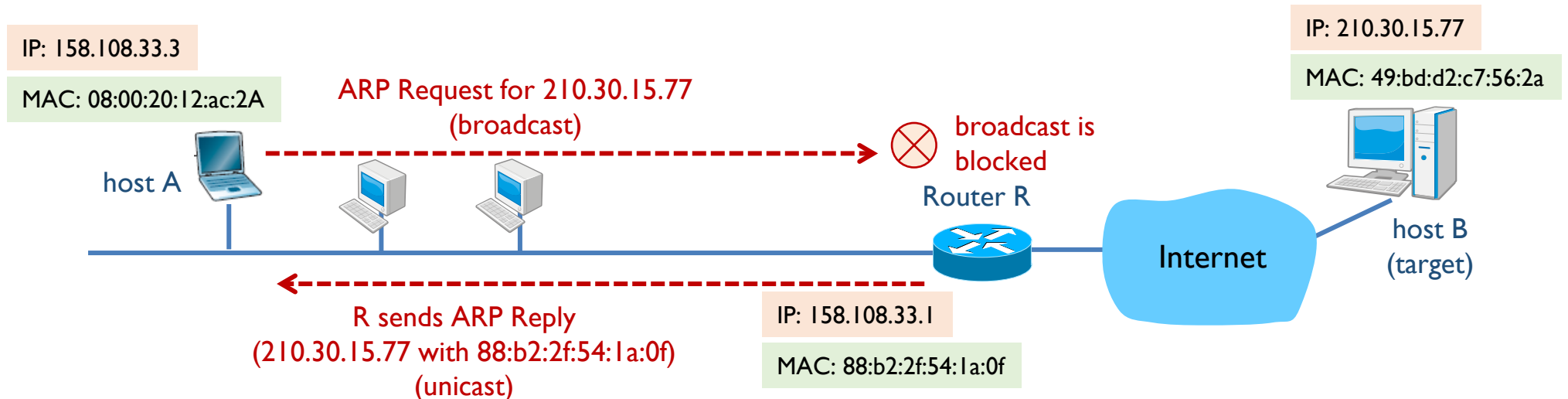
- host C – answers ARP reply message with its MAC address
- host B, C, and D – keep host A's IP/MAC addresses in its ARP table for future use



# Proxy ARP

## ❖ Proxy ARP

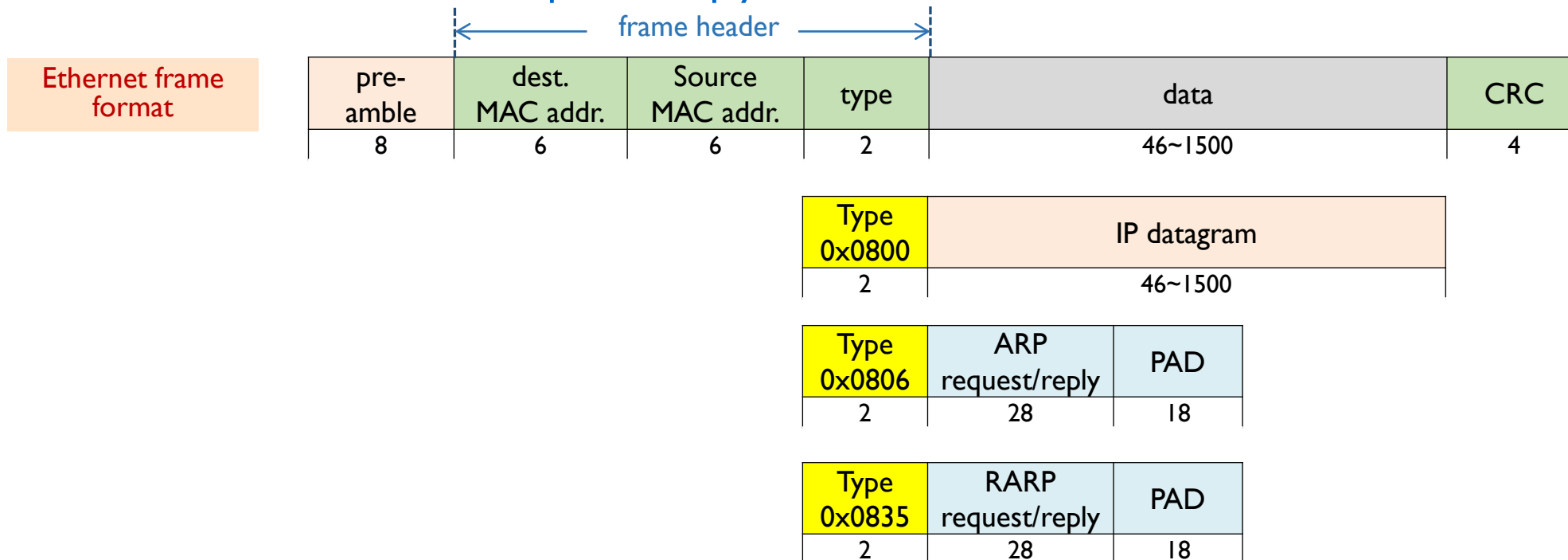
- Router blocks broadcasted ARP Request message
- Proxy ARP allows
  - the router to respond for remote host outside the local network
  - example
    - host A wants to communicate with host B, but knows only B's IP address



# ARP Message Format

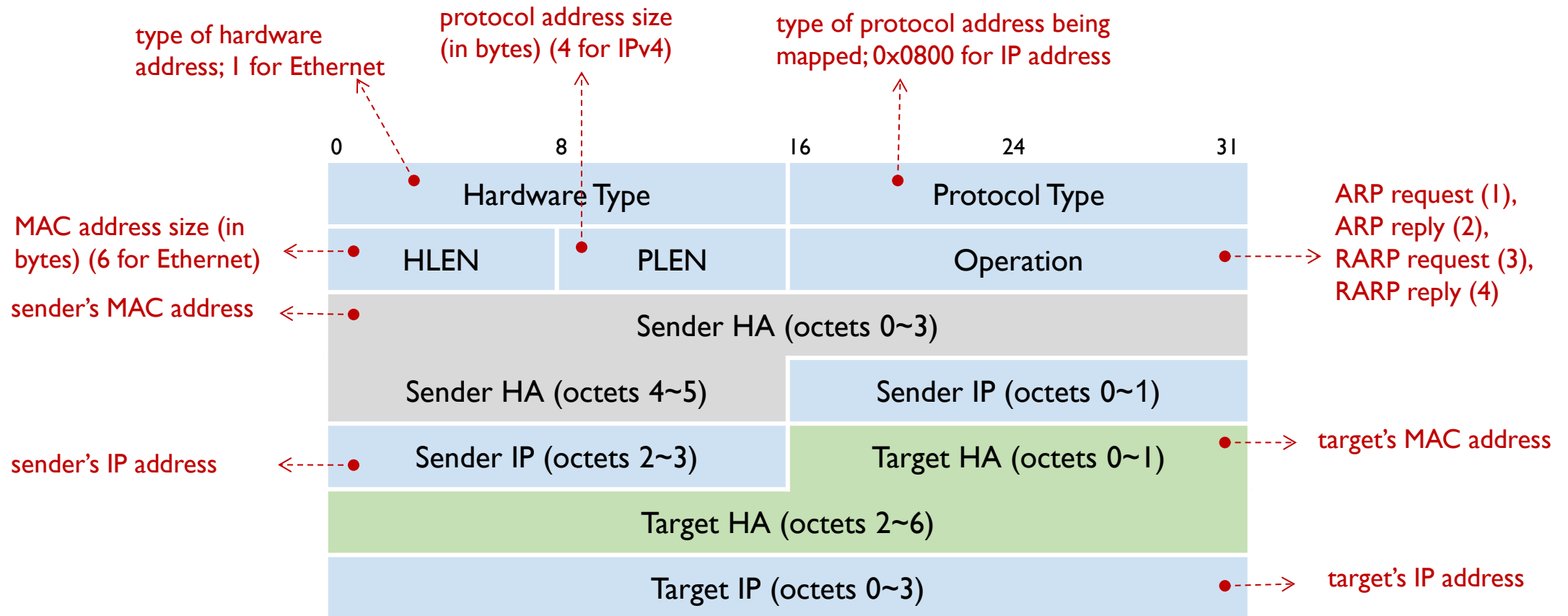
## ❖ Ethernet encapsulation

- type field
  - 0x0800 : IP datagram
  - 0x0806 : ARP Request / Reply
  - 0x0835 : RARP Request / Reply



# ARP Message Format

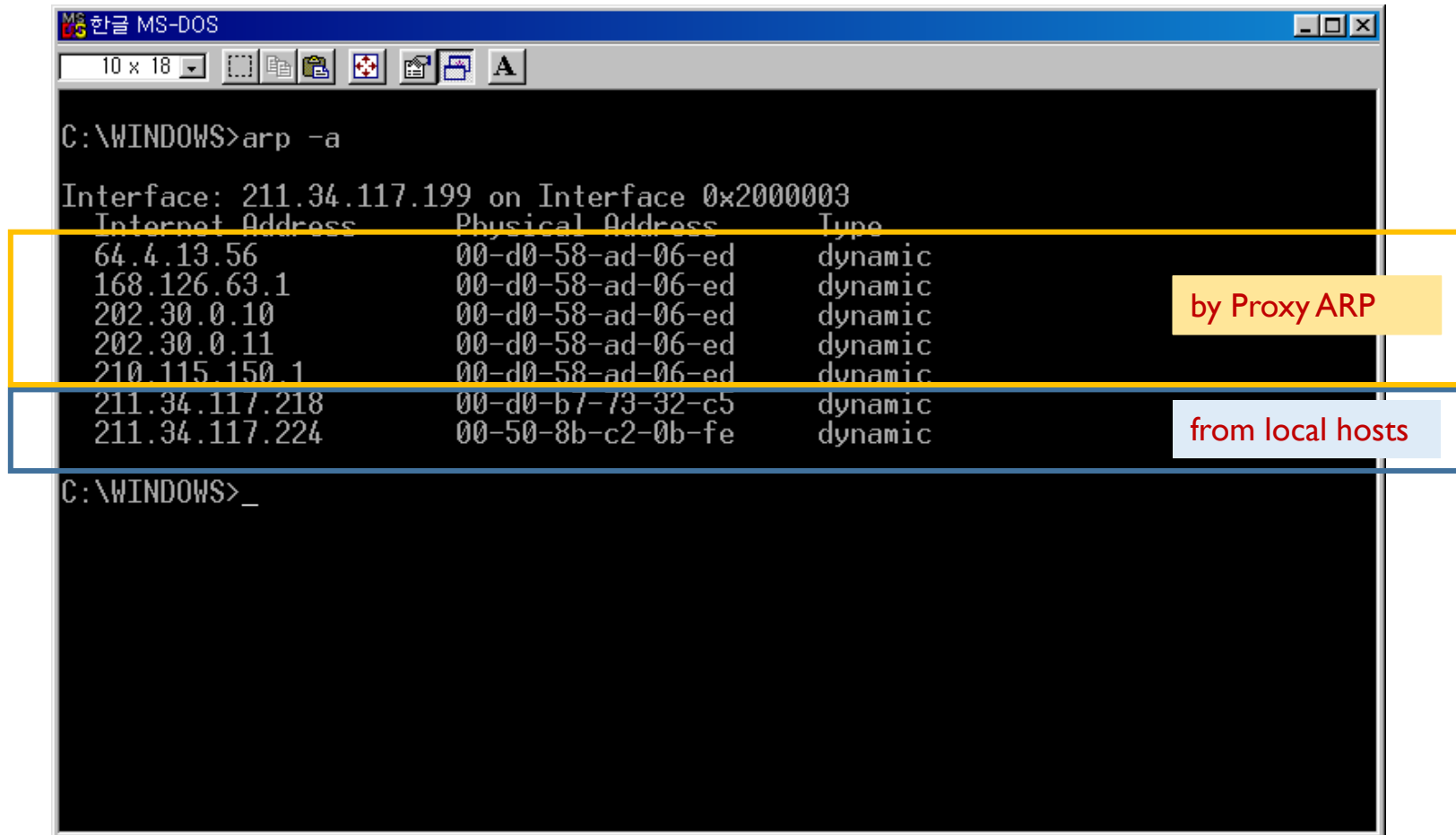
## ❖ ARP Message Format





# ARP Caching

## ❖ ARP cache



```
MS 한글 MS-DOS
10 x 18
C:\WINDOWS>arp -a

Interface: 211.34.117.199 on Interface 0x2000003
Internet Address      Physical Address      Type
64.4.13.56            00-d0-58-ad-06-ed    dynamic
168.126.63.1          00-d0-58-ad-06-ed    dynamic
202.30.0.10           00-d0-58-ad-06-ed    dynamic
202.30.0.11           00-d0-58-ad-06-ed    dynamic
210.115.150.1         00-d0-58-ad-06-ed    dynamic
211.34.117.218        00-d0-b7-73-32-c5    dynamic
211.34.117.224        00-50-8b-c2-0b-fe    dynamic

C:\WINDOWS>_
```

by Proxy ARP

from local hosts

# Example ARP Generation

## ❖ Definitions and Header

```
#define MAC_ADDR_LEN 6
#define IPv4_ADDR_LEN 4
#define ORIG_PACKET_LEN 64
#define ARPOP_REQUEST 1 // ARP Request
#define ARPOP_REPLY 2 // ARP Reply
#define ARPHRD_ETHER 1 // Ethernet
#define ETHERTYPE_IP 0x0800 // IP protocol
#define ETHERTYPE_ARP 0x0806 // ARP protocol
```

```
struct arp_message
{
    uint16_t ar_hrd; // Format of hardware address
    uint16_t ar_pro; // Format of protocol address
    uint8_t ar_hln; // Length of hardware address
    uint8_t ar_pln; // Length of protocol address
    uint16_t ar_op; // ARP opcode (command)
    uint8_t ar_sha[6]; // Sender hardware address
    uint32_t ar_sip; // Sender IP address
    uint8_t ar_tha[6]; // Target hardware address
    uint32_t ar_tip; // Target IP address
}
```

| Hardware Type          |      | Protocol Type          |
|------------------------|------|------------------------|
| HLEN                   | PLEN | Operation              |
| Sender HA (octets 0~3) |      |                        |
| Sender HA (octets 4~5) |      | Sender IP (octets 0~1) |
| Sender IP (octets 2~3) |      | Target HA (octets 0~1) |
| Target HA (octets 2~6) |      |                        |
| Target IP (octets 0~3) |      |                        |

# Example ARP Generation

## ❖ ARP packet generation (1/2)

```
void generate_packet_arp(u_char *packet, int op, u_char* smac, u_char* tmac, int *pklen)
{
    u_char                sendbuf[4096] = { 0 };
    struct eth_header     *eth;
    struct arp_message    *arph;
    uint32_t              src_ip=0, dst_ip=0;

    /* Prepare headers*/
    eth = (struct eth_header*)sendbuf;
    arph = (struct arp_message*)(sendbuf + sizeof(struct eth_header));

    // Ethernet header
    etype = ETHERTYPE_ARP;                // ethernet type for ARP
    get_header_ethernet(eth);

    // IP address manipulation
    inet_pton(AF_INET, sip, &src_ip);      // source IP address
    inet_pton(AF_INET, dip, &dst_ip);      // destination IP address
}
```

# Example ARP Generation

## ❖ ARP packet generation (2/2)

```
// ARP Message
arph->ar_hrd = htons(ARPHRD_ETHER);           // Format of hardware address
arph->ar_pro = htons(ETHERTYPE_IP);            // Format of protocol address
arph->ar_hln = MAC_ADDR_LEN;                   // Length of hardware address
arph->ar_pln = IPv4_ADDR_LEN;                  // Length of protocol address
arph->ar_op  = htons(op);                      // ARP opcode (1:request, 2:reply)

memcpy(arph->ar_sha, smac, MAC_ADDR_LEN);      // Sender hardware address
arph->ar_sip = src_ip;                         // Sender IP address
memcpy(arph->ar_tha, tmac, MAC_ADDR_LEN);      // Target hardware address
arph->ar_tip = dst_ip;                        // Target IP address

*pklen = sizeof(struct eth_header) + sizeof(struct arp_message);
if (*pklen <= ORIG_PACKET_LEN) *pklen = ORIG_PACKET_LEN;

memcpy(packet, sendbuf, sizeof(sendbuf));
}
```

# Example ARP Generation

```
uint8_t  dst_mac[6] = { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF };           // Broadcast MAC
uint8_t  src_mac[6] = { 0x14, 0x18, 0xc3, 0x7e, 0xdc, 0x0e };           // Source MAC

int main()
{
    ...
    pcap_findalldevs(&alldevs, errbuf);    // Retrieve the device list
    ...
    adhandle = pcap_open_live(d->name, 65536, 1, 1000, effbuf); // Open the adapter

    sip = "192.168.0.10";
    dip = "192.168.0.12";
    u_char  smac[6] = { 0x14, 0x18, 0xc3, 0x7e, 0xdc, 0x0e }; // sender mac address in ARP
    u_char  tmac[6] = { 0x00, 0x00, 0x00, 0x00, 0x00, 0x00 }; // target mac address in ARP

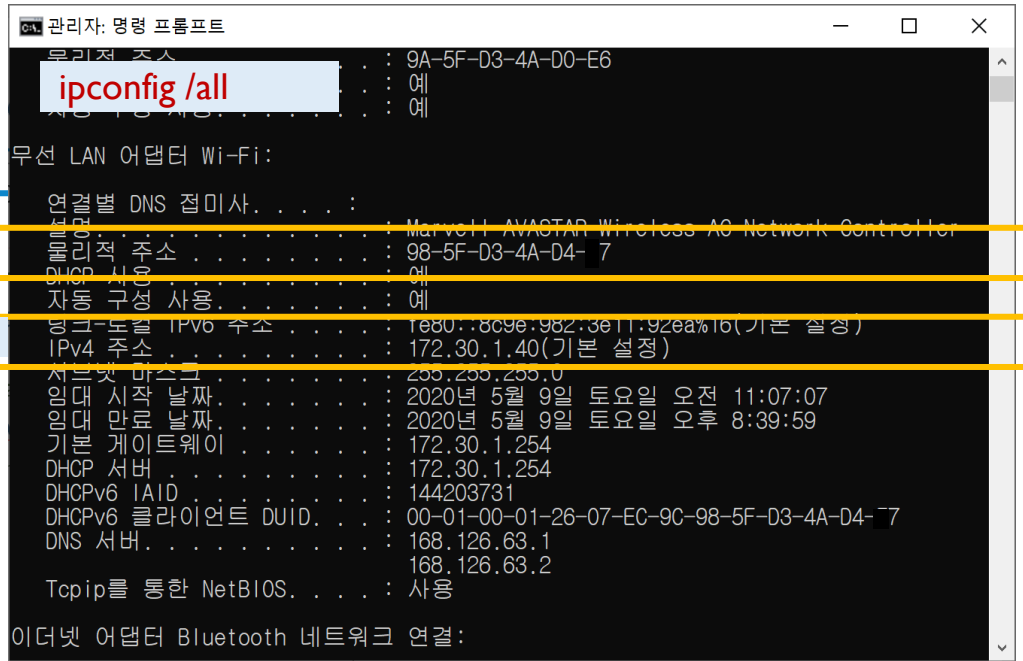
    generate_packet_arp(packet, arp_op, smac, tmac, &pklen);    // ARP Packet generation

    pcap_sendpacket(adhandle, packet, pklen);    // Send packet
}
```

# Example ARP Generation

## ❖ To run the example

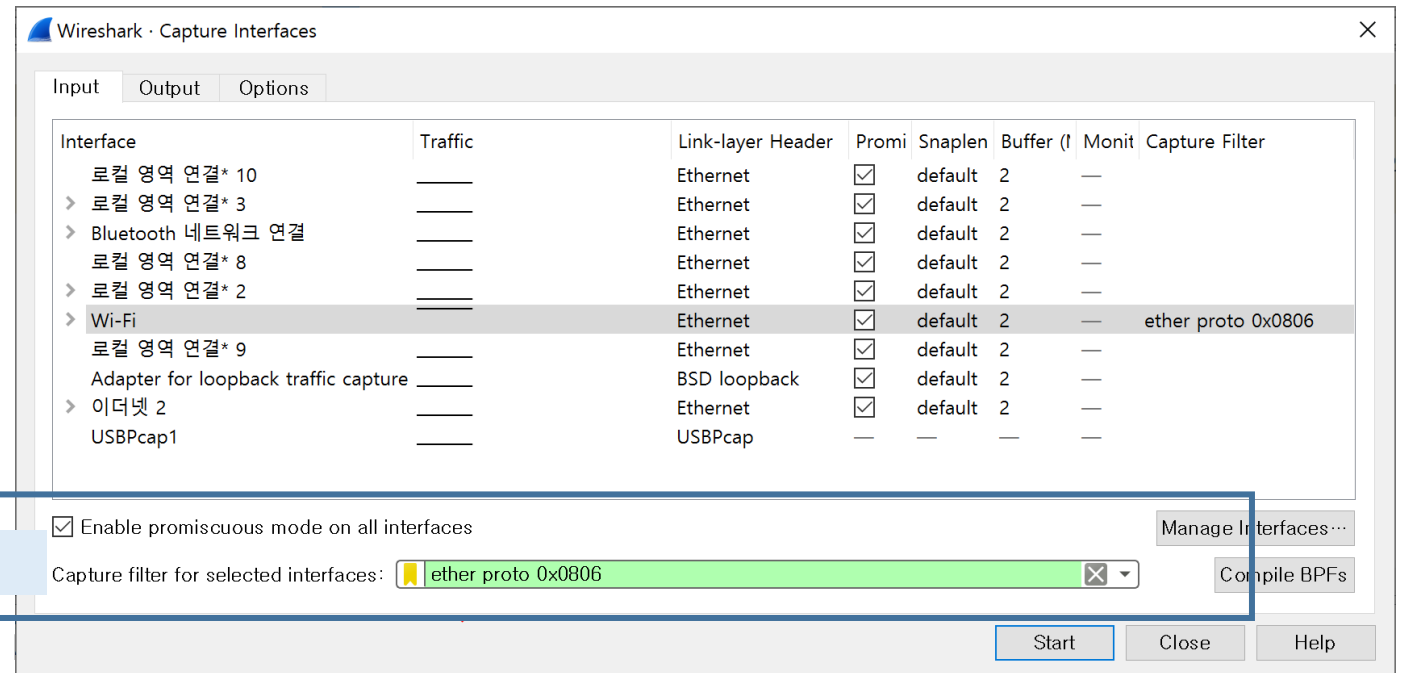
- check your ip and mac address
  - use “ipconfig /all” in command prompt
- flush ARP cache
  - “arp -d” in command prompt (as administrator mode)
- set filter in WireShark
  - Capture > Option
  - “ether proto 0x0806” (or arp)
- then
  - capture start (WireShark)
  - run the program



```
관리자: 명령 프롬프트
C:\>ipconfig /all

무선 LAN 어댑터 Wi-Fi:
. . . . .
MAC 주소 . . . . . : 98-5F-D3-4A-D4-7
IPv4 주소 . . . . . : 172.30.1.40(기본 설정)
. . . . .
. . . . .

이더넷 어댑터 Bluetooth 네트워크 연결:
. . . . .
MAC 주소 . . . . . : 98-5F-D3-4A-D4-7
IPv4 주소 . . . . . : 172.30.1.40(기본 설정)
. . . . .
. . . . .
```



filter option

# Example ARP Generation

```
Microsoft Visual Studio 디버그 콘솔

Index      Name                                     Description                                IPv4 address
[ 1] #Device#NPF_{27C9D449-0722-4B03-BFB6-A4F3DF51C58E} WAN Miniport (Network Monitor)
[ 2] #Device#NPF_{6281390D-492B-40B7-936C-A28CECDE84A4} WAN Miniport (IPv6)
[ 3] #Device#NPF_{7FBABBB3-1A92-4573-9581-BE701687333E} WAN Miniport (IP)
[ 4] #Device#NPF_{202AFD00-7BE4-4EC7-B823-1F4E97ED9EAE} Hyper-V Virtual Ethernet Adapter          172.26.48.1
[ 5] #Device#NPF_{6010B75C-166C-4E21-A43B-751BFE0F3BCF} Bluetooth Device (Personal Area Network) 169.254.56.228
[ 6] #Device#NPF_{1F9AE791-4F5F-4DCD-B8B5-B68037F79E19} VMware Virtual Ethernet Adapter for VMnet8 192.168.247.1
[ 7] #Device#NPF_{8CD4B3C8-85EC-4DC3-8D49-85D585666BD3} VMware Virtual Ethernet Adapter for VMnet1 192.168.17.1
[ 8] #Device#NPF_{316B5B54-81C9-4E86-94B0-E6480DCBCE0E} Intel(R) Wi-Fi 6 AX201 160MHz             192.168.0.10
[ 9] #Device#NPF_{Loopback} Adapter for loopback traffic capture      127.0.0.1
[10] #Device#NPF_{807D4AF4-003F-45BF-906A-F9383FA1B8CB} TAP-Win32 Adapter V9                     169.254.122.120
[11] #Device#NPF_{C60B3186-ABEA-411A-9C17-BCC10C50933C} TAP-NordVPN Windows Adapter V9           169.254.206.98
[12] #Device#NPF_{64ACF510-EEE7-4C38-9A9F-C0DBD65765FB} Realtek PCIe GbE Family Controller       210.107.197.10 169.254.98.142

Enter the interface number (1-12):8

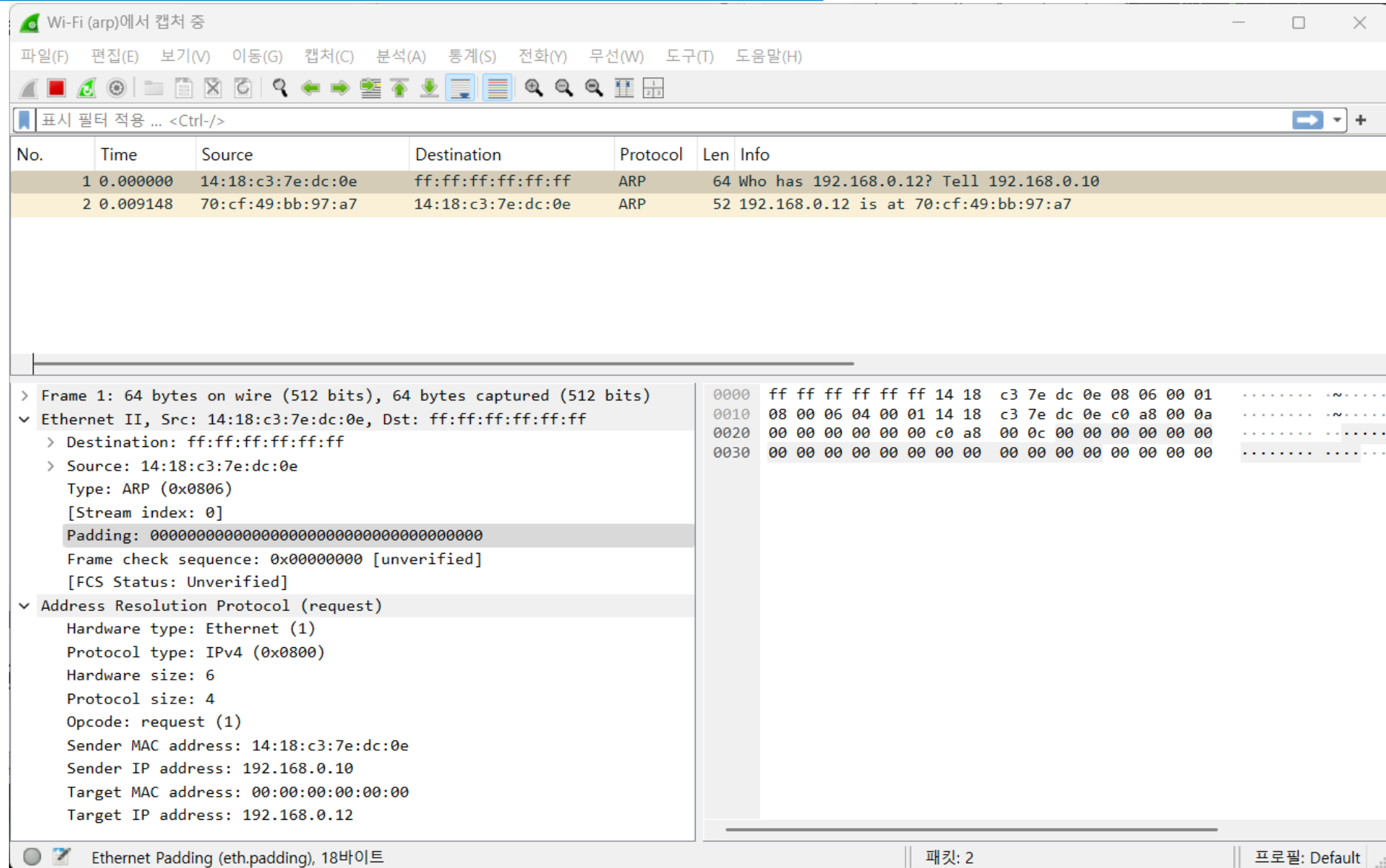
    Selected interface: #Device#NPF_{316B5B54-81C9-4E86-94B0-E6480DCBCE0E}, IP address: 192.168.0.10

Select the packet type (1:udp, 2:tcp, 3:icmp, 4:arp): 4
in main: ethype=0x0806 pklen=64
Packet sent successfully .....

C:\Users\User\SynologyDrive\Bhroh\VSPGM\C\CaptureFileAnalysis\#x64\Debug\CaptureFileAnalysis.exe(프로세스 49652)이(가) 0
코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

# Example ARP Generation

## ❖ Wireshark (1/2)



Wi-Fi (arp)에서 캡처 중

파일(F) 편집(E) 보기(V) 이동(G) 캡처(C) 분석(A) 통계(S) 전화(Y) 무선(W) 도구(T) 도움말(H)

표시 필터 적용 ... <Ctrl-/>

| No. | Time     | Source            | Destination       | Protocol | Len | Info                                    |
|-----|----------|-------------------|-------------------|----------|-----|-----------------------------------------|
| 1   | 0.000000 | 14:18:c3:7e:dc:0e | ff:ff:ff:ff:ff:ff | ARP      | 64  | Who has 192.168.0.12? Tell 192.168.0.10 |
| 2   | 0.009148 | 70:cf:49:bb:97:a7 | 14:18:c3:7e:dc:0e | ARP      | 52  | 192.168.0.12 is at 70:cf:49:bb:97:a7    |

> Frame 1: 64 bytes on wire (512 bits), 64 bytes captured (512 bits)  
▼ Ethernet II, Src: 14:18:c3:7e:dc:0e, Dst: ff:ff:ff:ff:ff:ff  
    > Destination: ff:ff:ff:ff:ff:ff  
    > Source: 14:18:c3:7e:dc:0e  
        Type: ARP (0x0806)  
        [Stream index: 0]  
        Padding: 00000000000000000000000000000000  
        Frame check sequence: 0x00000000 [unverified]  
        [FCS Status: Unverified]  
▼ Address Resolution Protocol (request)  
    Hardware type: Ethernet (1)  
    Protocol type: IPv4 (0x0800)  
    Hardware size: 6  
    Protocol size: 4  
    Opcode: request (1)  
    Sender MAC address: 14:18:c3:7e:dc:0e  
    Sender IP address: 192.168.0.10  
    Target MAC address: 00:00:00:00:00:00  
    Target IP address: 192.168.0.12

0000 ff ff ff ff ff ff 14 18 c3 7e dc 0e 08 06 00 01 ..... ~.....  
0010 08 00 06 04 00 01 14 18 c3 7e dc 0e c0 a8 00 0a ..... ~.....  
0020 00 00 00 00 00 00 c0 a8 00 0c 00 00 00 00 00 00 .....  
0030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....  
.....

Ethernet Padding (eth.padding), 18바이트      패킷: 2      프로필: Default



# Example ARP Generation

## ❖ Wireshark (2/2)

The image shows a Wireshark capture window titled "Wi-Fi (arp)에서 캡처 중". The packet list shows two packets:

| No. | Time     | Source            | Destination       | Protocol | Len | Info                                    |
|-----|----------|-------------------|-------------------|----------|-----|-----------------------------------------|
| 1   | 0.000000 | 14:18:c3:7e:dc:0e | ff:ff:ff:ff:ff:ff | ARP      | 64  | Who has 192.168.0.12? Tell 192.168.0.10 |
| 2   | 0.009148 | 70:cf:49:bb:97:a7 | 14:18:c3:7e:dc:0e | ARP      | 52  | 192.168.0.12 is at 70:cf:49:bb:97:a7    |

The packet details pane shows the structure of the selected packet (Frame 2):

- Frame 2: 52 bytes on wire (416 bits), 52 bytes captured (416 bits)
- Ethernet II, Src: 70:cf:49:bb:97:a7, Dst: 14:18:c3:7e:dc:0e
  - Destination: 14:18:c3:7e:dc:0e
  - Source: 70:cf:49:bb:97:a7
  - Type: ARP (0x0806)
  - [Stream index: 1]
  - Trailer: 00000000000000000000
- Address Resolution Protocol (reply)
  - Hardware type: Ethernet (1)
  - Protocol type: IPv4 (0x0800)
  - Hardware size: 6
  - Protocol size: 4
  - Opcode: reply (2)
  - Sender MAC address: 70:cf:49:bb:97:a7
  - Sender IP address: 192.168.0.12
  - Target MAC address: 14:18:c3:7e:dc:0e
  - Target IP address: 192.168.0.10

The packet bytes pane shows the raw data in hexadecimal and ASCII:

```
0000 14 18 c3 7e dc 0e 70 cf 49 bb 97 a7 08 06 00 01  ....p- I....
0010 08 00 06 04 00 02 70 cf 49 bb 97 a7 c0 a8 00 0c  ....p- I....
0020 14 18 c3 7e dc 0e c0 a8 00 0a 00 00 00 00 00 00  ....
0030 00 00 00 00  ....
```

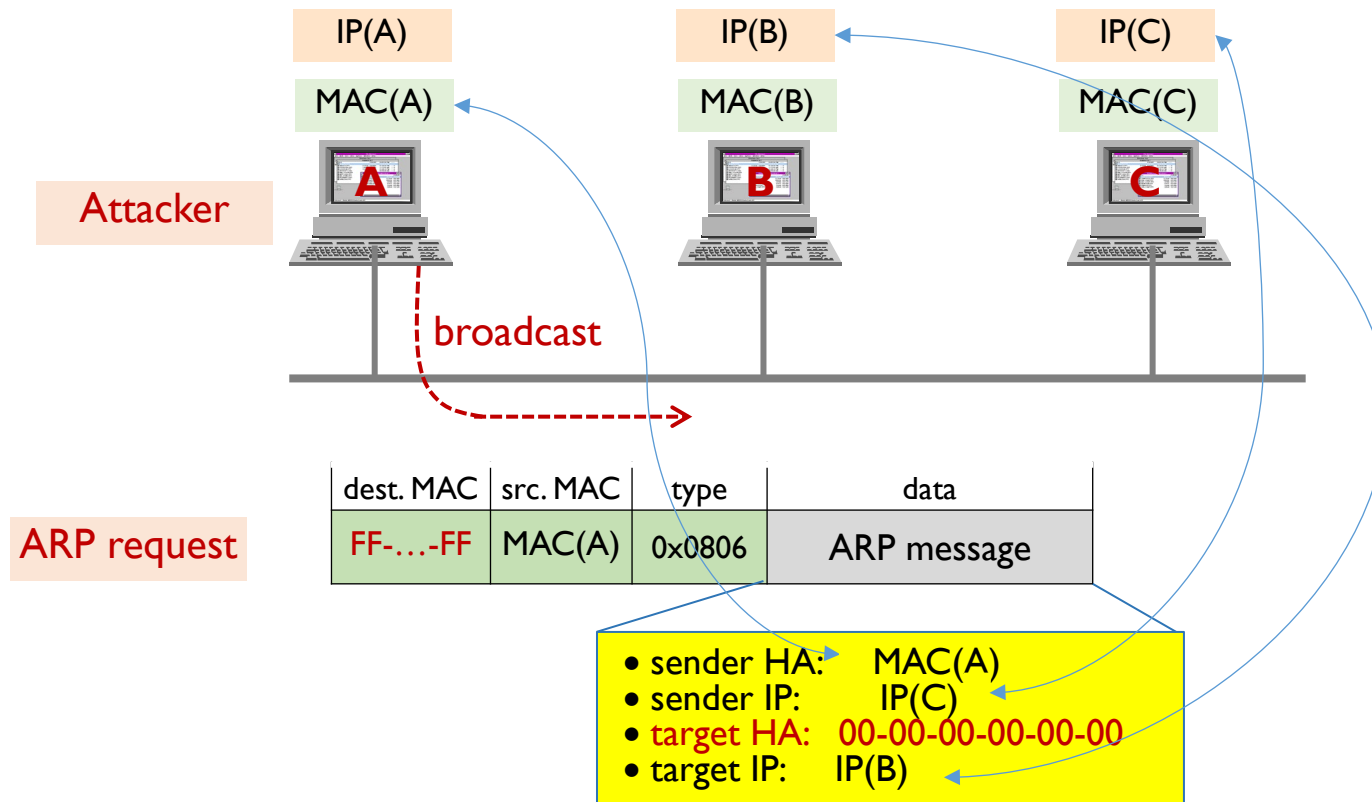
The status bar at the bottom indicates "Wi-Fi: <live capture in progress>" and "패킷: 2" (Packet: 2).



# PRACTICE – ARP POISONING

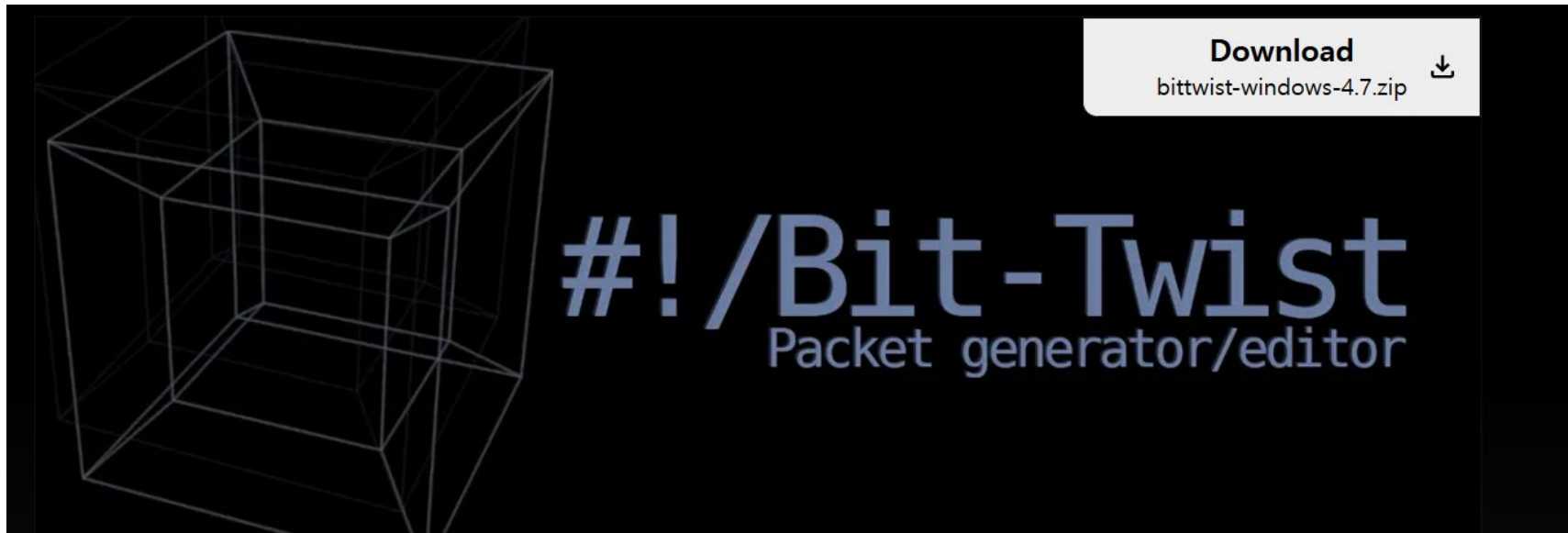
# ARP Poisoning

❖ How does it work ?



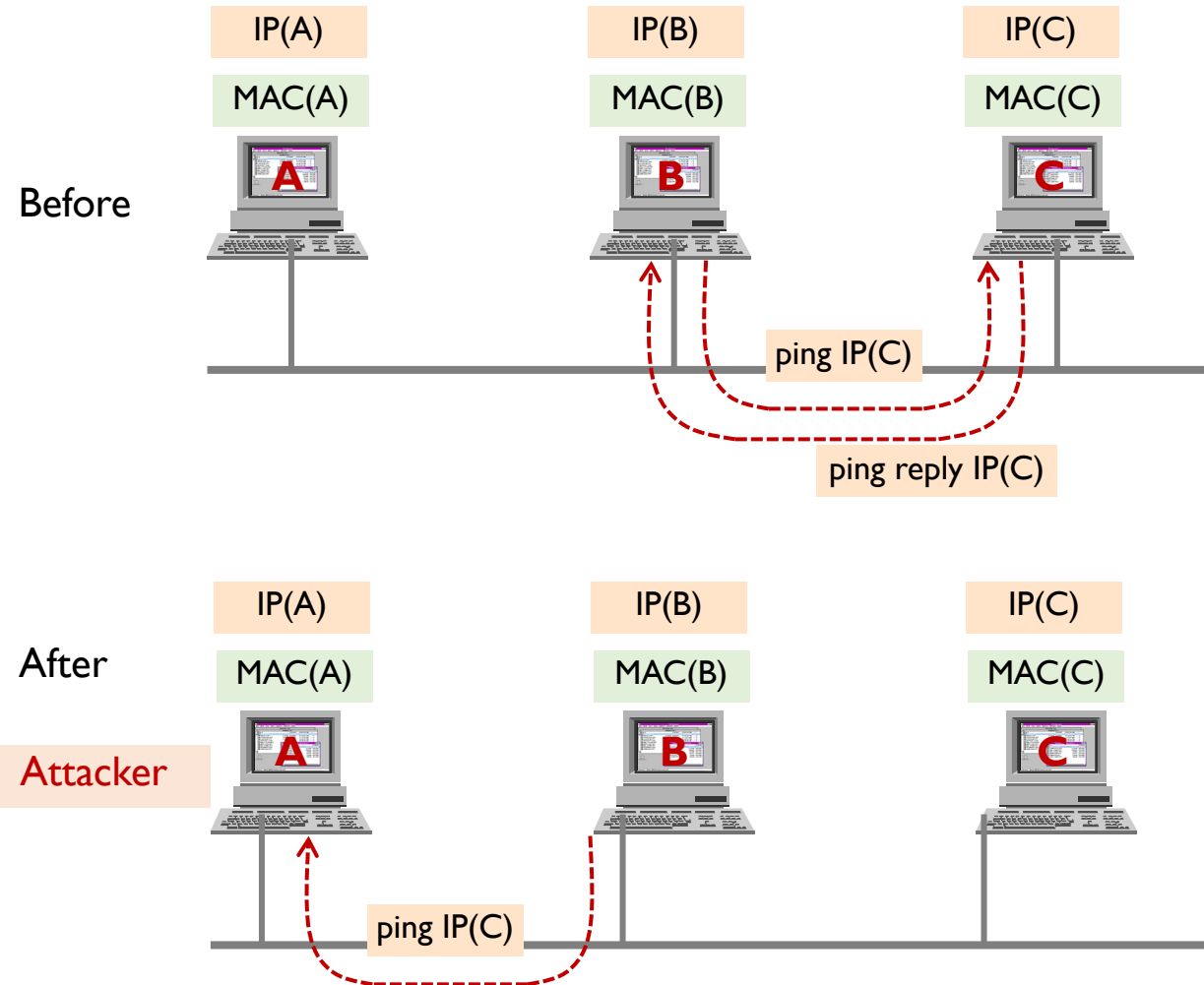
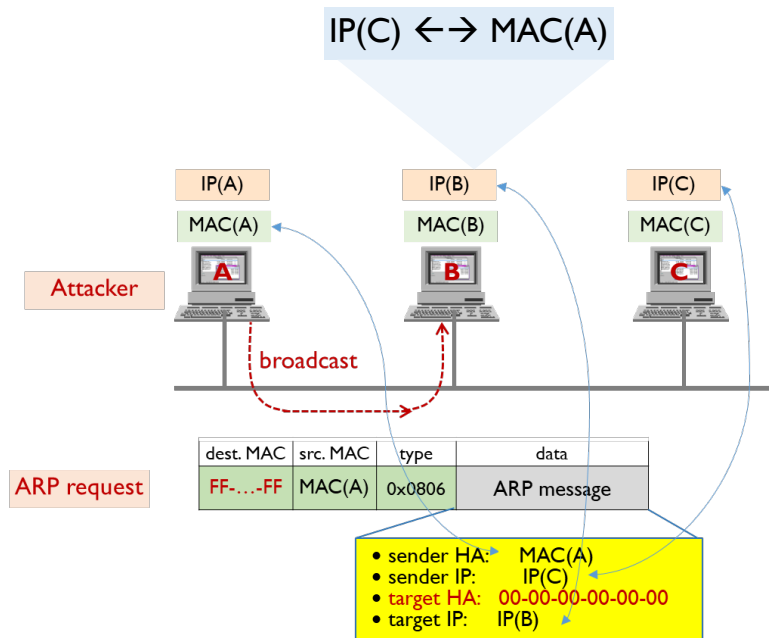
## ❖ Bit-Twist

- <https://bittwist.sourceforge.io/index.html>



# ARP Poisoning

## ❖ How does it work ?



# Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN



Mobile **Multimedia Convergence** Network Lab.

Homepage: <http://mmcn.ajou.ac.kr>

facebook: <http://www.facebook.com/#!/groups/190702010947314>

